CellSecInspector: Safeguarding Cellular Networks via Automated Security Analysis on Specifications

Ke Xie[1], Xingyi Zhao[1], Min-Yue Chen[2], Yu-An Chen[2], Yiwen Hu[3], Munshi Saifuzzaman[1], Wen Li[1], Shuhan Yuan[1], Guan-Hua Tu[2], Tian Xie[1]
[1] Utah State University [2] Michigan State University [3] University of Maryland, Baltimore County
{ke.xie, xingyi.zhao, munshi.saifuzzaman, awen.li, Shuhan.Yuan, tian.xie}@usu.edu
{chenmi33, cheny132, ghtu}@msu.edu huyiwen@umbc.edu

arXiv:2512.24682v4 [cs.CR] 7 Sep 2026

Abstract

The complexity, interdependence, and rapid evolution of 3GPP specifications present fundamental challenges for ensuring the security of modern cellular networks. Manual reviews and existing automated approaches, which often depend on rule-based parsing or small sets of manually crafted security requirements, fail to capture deep semantic dependencies, cross-sentence/clause relationships, and evolving specification behaviors. In this work, we present *CellSecInspector*, an automated framework for security analysis of 3GPP specifications. *CellSecInspector* extracts structured state–condition–action (SCA) representations, models mobile network procedures with comprehensive function chains, systematically validates them against 9 foundational security properties under 4 adversarial scenarios, and automatically generates test cases. This end-to-end approach enables the automated discovery of vulnerabilities without relying on manually predefined security requirements or rules. Applying *CellSecInspector* to the well-studied 5G and 4G NAS and RRC specifications and selected sections of TS 23.501 and TS 24.229, it discovers 43 vulnerabilities, 7 of which are previously unreported. Our findings show that *CellSecInspector* is a scalable, adaptive, and effective solution to assess 3GPP specifications for safeguarding operational and next-generation cellular networks.

CCS Concepts

• **Security and privacy → Mobile and wireless security**.

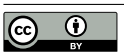
ACM Reference Format:
Ke Xie[1], Xingyi Zhao[1], Min-Yue Chen[2], Yu-An Chen[2], Yiwen Hu[3], Munshi Saifuzzaman[1],, Wen Li[1], Shuhan Yuan[1], Guan-Hua Tu[2], Tian Xie[1] . 2026. *CellSecInspector*: Safeguarding Cellular Networks via Automated Security Analysis on Specifications. In *The 32nd Annual International Conference on Mobile Computing and Networking (MobiCom '26), October 26–30, 2026, Austin, TX, USA.* ACM, New York, NY, USA, 25 pages. https://doi.org/10.1145/3795866.3844474

1 Introduction

Cellular networks are critical for modern global connectivity, with 5G and 4G forming the backbone of modern mobile communication systems. Technical specifications and architectures are standardized by the 3rd Generation Partnership Project (3GPP), an association of seven Organizational Partners, including ATIS in the U.S., ETSI in Europe, and CCSA in China. Thousands of specifications are published by 3GPP to ensure global interoperability across mobile networks and devices. Consequently, mobile equipment vendors and operators are required to ensure compliance with 3GPP specifications. However, this standardization introduces systemic risks: *Design flaws in 3GPP specifications can propagate into all compliant networks and devices, creating widespread threats with global negative impact.* Moreover, the increasing complexity, frequent revisions, and evolving new generations of mobile networks continue to expand attack surfaces.

To mitigate those risks, researchers spend increasing effort to scrutinize cellular network specifications and services documented in the 3GPP specifications to identify design issues. However, the traditional manual analysis by experts is slow, error-prone, and infeasible at scale. State-of-the-art automated approaches primarily rely on rule-based approaches (e.g., dependency parsing [14, 25]), and keyword-driven scanning (e.g., hazard indicators [25]), failing to capture the deeper and implicit knowledge in 3GPP specifications. As a result, vulnerabilities in critical mobile network procedures continue to be exploited, enabling attacks such as downgrade attacks [15, 22, 25, 42, 44, 46, 48, 50, 51, 64]. These methods suffer from three fundamental limitations: (1) they lack deep insights into the logic, processes, and context of specification texts, particularly when dealing with cross-sentence/clause dependencies; (2) their security requirements are manually developed for specific procedures, which severely limits scalability across the thousands of 3GPP specifications; (3) their analysis pipelines require costly manual updates to remain synchronized with evolving 3GPP releases. These limitations highlight the urgent need for an intelligent, automated system capable of understanding 3GPP specifications and systematically assessing them.

In this study, we develop *CellSecInspector* (Cellular Network Security Inspector), an automated framework capable of understanding 3GPP specifications and systematically assessing them via foundational security properties. *CellSecInspector* models mobile networks in structured SCA (state-condition–action) representations from 3GPP specifications, and examines them with foundational security properties. By integrating specification analysis with automated test case generation, *CellSecInspector* delivers a scalable, adaptive, and future-proof solution for 3GPP specification security assessment. In this study, *CellSecInspector* analyzes the critical NAS and RRC layers for current operational 5G and 4G mobile networks as well as the selected sections of TS 23.501 and TS 24.229. *CellSecInspector* not only discovers 36 previously reported vulnerabilities but



This work is licensed under a Creative Commons Attribution 4.0 International License.
MobiCom '26, Austin, TX, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2505-0/2026/10
https://doi.org/10.1145/3795866.3844474

also reports 7 new vulnerabilities. We summarize our contributions as follows.

◇**New Framework:** We developed *CellSecInspector*, a novel approach for automated security assessment of 3GPP specifications, advancing toward fully automated and scalable specification analysis for safeguarding not only operational 5G/4G cellular networks but also Next-G mobile networks.

◇**New Findings:** Applying *CellSecInspector* to 5G and 4G NAS and RRC specifications, as well as the selected sections of TS 23.501 and TS 24.229, we discovered 43 vulnerabilities, 7 of which were previously unreported. They are shown to have serious security consequences, demonstrating the urgent need for automated 3GPP specification analysis and the effectiveness of *CellSecInspector* in identifying known and previously undetected vulnerabilities.

2 Background

Non-Access Stratum (NAS) and Radio Resource Control (RRC): The NAS is a key component of the control plane in modern 5G and 4G cellular networks, managing mobility, session control, and security between user equipment (UE) and the core network. It ensures seamless session establishment, authentication, and mobility across different networks. The RRC operates within the Access Stratum (AS) of both 5G and 4G networks and manages signaling between the UE and the radio access network (RAN). It handles connection setup, maintenance, release, handovers, paging, measurement reporting, and security configuration.

Large Language Models (LLMs): LLMs have emerged as powerful systems trained on massive text corpora to perform a wide range of natural language processing tasks, including comprehension, reasoning, and generation. Modern LLMs, including ChatGPT [21] and DeepSeek [17], have demonstrated impressive emergent abilities across diverse tasks, showing that LLMs can go beyond text processing to support domain-specific reasoning. This capability makes LLMs a promising foundation for analyzing complex 3GPP specifications, where precision, contextual understanding, and reasoning across specifications are critical.

3 Related Work

Given the scale and complexity of 3GPP specifications, NLP techniques have increasingly been applied for cellular network security analysis. *Atomic* [25] develops adversarial test cases from the 4G NAS specification. It relies heavily on keyword seeds (e.g., conditional words, risky operations) to identify relevant sentences to generate the test cases. *ConTester* [24] automates LTE NAS conformance test generation by constructing an Event Dependency Graph (EDG). Its scope is limited to generating test cases for assessing commercial mobile devices rather than analyzing design issues within 3GPP specifications. *CellularLint* [61] leverages a domain-adapted BERT [33] model to identify inconsistencies in 5G and 4G NAS and security-related specifications. Its emphasis is on sentence-level semantic conflicts, similar to natural language inference (NLI) tasks [20, 30], rather than systematic security analysis. *Hermes* [14] parses the specification via handcrafted grammars using constituency parsing and dependency parse trees to develop FSMs, which are further manually translated into SMV language [14, 28, 42, 44] for formal security analysis. It remains rule-intensive and lacks flexibility to adapt to evolving specifications. *ARCANE* [66] leverages large language models to assist specification understanding and combines it with passive model learning to automatically construct FSMs for model-based fuzzing. The generated models are then used to produce fuzzing inputs for testing 5G O-RAN implementations. However, ARCANE focuses on implementation bug discovery rather than systematic analysis of design issues in 3GPP specifications. In contrast, *CellSecInspector* provides a security analysis framework that aims to comprehensively comprehend 3GPP specifications and conduct systematic analysis of cellular network specifications. It can be easily adapted to conduct security analysis of large-scale and evolving 3GPP specifications, providing an efficient and effective approach to complement traditional manual efforts. Table 7 in Appendix B compares these works.

4 Overview of *CellSecInspector*

4.1 Approach Skeleton

Figure 1 shows the overview of *CellSecInspector*, which can automatically comprehend complicated 3GPP specifications, model the mobile network functions using structured state-condition-action (SCA) representations, and examine whether the 3GPP-designed procedures can violate foundational security properties. Five primary modules of *CellSecInspector* are developed: *SpecAdaptation* (Specification Adaptation), *SCA Representation Extractor*, *Function Chain Builder*, *SecOracle* (Security Oracle), and *VulnTestGenerator* (Vulnerability Test Generator). *SpecAdaptation* adapts the core model *CellSecInspector-Core* to the cellular network domain knowledge using the collected and curated 3GPP specifications. *CellSecInspector-Core* is further used in later modules. Two components are involved to comprehensively model the complicated mobile network functions. *SCA Representation Extractor* parses and extracts the SCA nodes from 3GPP specifications and the internal domain knowledge learned via *SpecAdaptation*. *Function Chain Builder* connects scattered SCA nodes into function chains for comprehensively modeling the cellular network services specified in 3GPP specifications. Next, *SecOracle* conducts the security checking using the foundational security properties. Lastly, *VulnTestGenerator* automatically generates test cases for candidate violations.

4.2 Challenges and Insights

3GPP specifications are inherently complex and have a vast scope, complicating the development of a framework for comprehending the cellular networks governed by 3GPP specifications. For instance, NAS and RRC specifications span thousands of pages, with intricate interdependencies and frequent updates across releases. A single 5G NAS specification TS 24.501 contains over 981 pages [10]. Reading and understanding 3GPP specifications is a time-consuming challenge even for experts. We design and develop *CellSecInspector* based on a pretrained LLM. To enable *CellSecInspector* to discover vulnerabilities from 3GPP specifications, we solve three main challenges.

C1: Lack of annotated resources for mobile network security research tasks. Different from common NLP (natural language processing) research tasks, such as text classification, that come with the thorough and well-annotated datasets, there are no well-known and easily accessible resources in the mobile network research

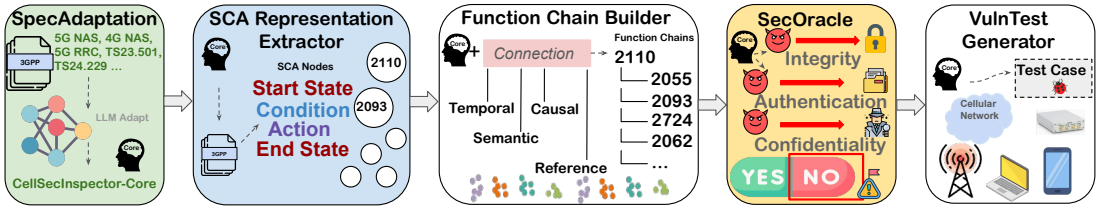


Figure 1: *CellSecInspector* Overview

community. Specifically, we need to model the complex mobile network procedures first to conduct the security analysis of mobile network functions and services. Unfortunately, there are no established and public annotation specifications or training resources in this domain-specific task.

◇ **Insight1:** To address C1, we propose the *SCA Representation Extractor*, which is designed to extract information for modeling the mobile network functions and services from 3GPP specifications. Two approaches are developed. First, instead of constructing the traditional FSM (finite state machine) models, *SCA Representation Extractor* captures the self-contained transition of mobile network procedures in our proposed *SCA (State-Condition-Action)* structure that can compose the complex and completed function event in a single presentation. Second, we adopt the efficient few-shot In-Context Learning (ICL) to enable our system to extract SCA nodes without large-scale annotated datasets.

C2: Computational cost for linking SCA nodes at scale. After deriving scattered SCA nodes, these nodes must be linked to construct complete mobile network procedures and capture the comprehensive network status (e.g., whether the security context is established).

However, linking SCA nodes at scale is challenging because different procedures described in SCA nodes may or may not share meaningful relationships. Moreover, to avoid redundancy, 3GPP specifications use cross-references between clauses and specifications, further complicating the linkage process. With thousands or more SCA nodes that can create millions of candidate connections in a quadratic manner, it imposes substantial computational overhead for linking SCA nodes at scale.

◇ **Insight2:** To address C2, we introduce the *Function Chain Builder*, which integrates two approaches: Type I: Node-Informed Exhaustive Connection and Type II: Reference Guided Connection. Type I compares all node pairs to identify three types of valid transitions: temporal connections that link nodes with identical end and start states, semantic connections that capture equivalent but textually different states, and causal connections where one node's state, condition, or action enables another. While Type I provides comprehensive coverage, its $O(n^2)$ complexity is costly as the number of nodes grows. To reduce this burden, Type II leverages the rich cross-references embedded in 3GPP specifications (e.g., '*as specified in subclause X.Y.Z*') to restrict the search space to relevant sections. Together, these strategies enable efficient and automated construction of function chains across 3GPP specifications.

C3: Manually crafted security requirements are impractical for security assessment of large-scale mobile network functions. State-of-the-art methods, such as *Hermes* [14] and *ConTester* [24], rely on a small set of manually crafted security requirements for specific procedures. For example, ConTester's security requirement S6 mandates that '*Except the messages listed below, no NAS signaling messages shall be processed by the receiving EMM entity in the UE forwarded to the ESM entity, ...*', which is the sentence directly from TS 24.301 subclause 4.4.4.2. When new NAS procedures are introduced in subsequent 3GPP releases, if these sentences are updated, their security requirements must be manually revised, limiting the scalability for the security assessments of large-scale mobile network functions. Similarly, approaches that build FSM models also need to manually translate FSM models into SMV language [14, 28, 42, 44] to enable formal security analysis, requiring significant expert efforts that are not practical given the complexity and breadth of modern mobile networks.

◇ **Insight3:** We develop the *SecOracle* that introduces 9 foundational security properties. *Unlike procedure-specific requirements, these properties are derived from classic and foundational security principles applicable across the entire mobile network stack.* Therefore, *SecOracle* can operate on large-scale cellular network procedures with broader security checkpoints. This equips *CellSecInspector* with the foundational ability to detect previously unreported vulnerabilities.

5 Detailed Design

We next present *CellSecInspector* in detail. Additional implementations and formalized algorithms are provided in Appendix C.

5.1 *SpecAdaptation*

We develop the core reasoning engine, *CellSecInspector-Core*, on top of open-source LLMs, in *SpecAdaptation* to capture the cross-sentence and cross-clause relationships from the complex 3GPP specifications. To enable it to generalize and reason in 3GPP specifications, we expose *CellSecInspector-Core* to broad information to obtain cellular specification domain knowledge in the following two steps.

Cross-sentence/clause relation preserving data sanitation. To ensure *CellSecInspector-Core* can be exposed to the broad cellular network domain knowledge, we begin by collecting and curating 5G and 4G technical specifications from the 3GPP portal [7] as the corpus for training. To ensure consistency and clarity, we sanitize the specifications by (a) removing redundant spaces, extraneous line breaks, and leading symbols (e.g., dots, bullets, hyphens), (b) trimming trailing punctuation, (c) discarding empty parentheses and special unicode characters, and (d) removing tables and figures, which are usually explained or demonstrated in texts. Note that references (e.g., 'as specified in subclause X.Y.Z') are preserved in

the datasets, ensuring *CellSecInspector-Core* can learn the cross-sentence and cross-clause relations via explicit references that are neglected by state-of-the-art works [61].

Domain knowledge adaptation. A straightforward approach to expose standards knowledge to *CellSecInspector-Core* is to continually train (i.e., pretrain) the model on 3GPP specifications. While feasible in principle, this approach has two key limitations. First, pretraining a model is computationally expensive and difficult to maintain for the fast-evolving standards. 3GPP standards evolve frequently across different releases and versions. A continuously trained model can become out-of-date quickly unless training is performed repeatedly. Second, training an LLM on a narrow domain corpus (i.e., specifications) under limited compute and data diversity can degrade the model's general reasoning and instruction-following abilities [37, 47, 56]. For example, a continually pretrained model may exhibit catastrophic forgetting and spec-confusion behaviors, such as conflating procedures across different releases, hallucinating missing steps, or recurring boilerplate in specifications. These failures are harmful for standards analysis, where correctness hinges on precise conditions, message ordering, and release-specific normative requirements.

In our design, we leverage Retrieval-Augmented Generation (RAG) [52] to adapt *CellSecInspector-Core* to domain knowledge without modifying the base model. Specifically, *CellSecInspector-Core* retrieves relevant specification excerpts and conditions its generation on them, using the retrieved text as grounded evidence to complement its parametric knowledge. The retrieval corpus is organized as versioned specification chunks, where each chunk preserves the source specification number, release/version, clause identifier, and cross-reference information. Concretely, given a query (e.g., message type, timer, or a security requirement), it searches a versioned corpus of 3GPP documents, which is prepared in the aforementioned step, and attaches the top-matched paragraphs (including section and release/version identifiers) to the model prompt. This retrieval step provides two technical benefits: (i) it ensures that *CellSecInspector-Core* uses the explicitly selected release/version text, and (ii) it reduces hallucination by requiring outputs to be justified by retrieved normative statements rather than relying solely on the model's memory. Finally, *CellSecInspector-Core* obtains the 3GPP specification domain knowledge, and it will be further used in all modules. The detailed workflow is provided in Algorithm 1 in Appendix C.

5.2 *SCA Representation Extractor*

To address C1 and comprehensively model the complicated cellular network functions, we develop the *SCA (State-Condition-Action) Representation Extractor* that extracts the structured function representations in SCA nodes from 3GPP specifications. Each SCA node contains four fields: the start state, representing the initial state before the transition; the condition, denoting the triggering clause or prerequisite specified in the text; the action, describing the operation mandated by the specification upon satisfaction of the condition; and the end state, indicating the resulting state after the action is executed.

SCA node vs. FSM state. Different from the traditional finite state machine (FSM) that separates the states and transitions and stores the transition logic on edges between states, the SCA representation structure (i.e., node) aims to directly capture the self-contained transition, binding the logic of condition and action. This structure introduces two main advancements to maximize the utilization of the text processing capability of *CellSecInspector-Core*. First, it presents explicit transition semantics, including where the transition starts, when it is valid (condition), what happens (action), and where it ends. Each SCA node is one self-contained transition. The explicit viewpoint of a transition saves the step of merging all involved edges in the FSM when transitioning from one state to another. That is, an FSM state may have several edges with its connected states, requiring extra effort to identify how to transition to the next state. Second, it allows complex conditions and actions, combining internal variables, timers, and status, to be described in a single node, which is valuable for modeling cellular network functions where the transitions may depend on complicated conditions and actions. *Note that SCA nodes can be easily transformed into the FSM states and edges. However, it is challenging to transform them in reverse.*

Few-shot training with minimal dataset development overhead. To extract SCA nodes from 3GPP specifications, instead of developing a large dataset of many SCA nodes training cases with heavy manual engineering overhead, we leverage efficient in-context learning (ICL) [21] to educate *CellSecInspector* without time-consuming fine-tuning to update model parameters. ICL works by embedding carefully selected demonstration examples and task-specific instructions directly into the model's input context, allowing the model to infer the desired output format. This method enables *CellSecInspector* to generalize to the SCA extraction task with a limited number of examples. Specifically, we instruct *CellSecInspector* to extract structured specification events from 5G and 4G specifications by mapping each relevant sentence into an SCA node. Along with the instruction, a few manually developed SCA nodes are given for *CellSecInspector* to infer. From each specification, we develop two examples to demonstrate common specification patterns. Sentences that accurately describe function events are expected to populate all four fields. If a sentence contains cross-references (e.g., 'as specified in subclause X.Y.Z'), the referenced content is retained verbatim in the most relevant field. For ambiguous or underspecified sentences, we employ best-effort inference. If no definition is found, the corresponding field is marked as 'Not explicitly defined' to ensure completeness of the extracted structure. Figure 20 in Appendix C shows an example ICL instruction.

SCA node example. Figure 2 presents an extracted SCA node from a short description in 5G NAS specification TS 24.501 by the following steps. *CellSecInspector* first parses the sentence structure using domain knowledge learned in *SpecAdaptation*. There was no explicit start state in this given example. *CellSecInspector* identifies the triggering condition by detecting the phrase upon receipt of a message, recognizing that the transition is activated when the UE receives a message for a network-initiated 5GMM common procedure. Next, *CellSecInspector* extracts the actions from the phrases '*shall stop timer T3540*' and '*respond to the procedure*,' which are organized as the structured expression that covers these two operations: stop T3540 **and** respond via the existing N1 NAS signalling connection as specified in subclause 5.4. From the derived condition and action, including '*via the existing N1 NAS signalling connection*' and '*timer*

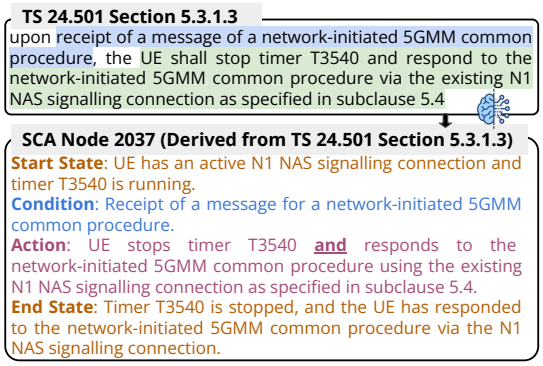


Figure 2: SCA representation for Node 2037

T3540', it is inferred that the UE already maintains an active N1 NAS signalling connection and T3540 is running before the event occurs, which are captured as the start state. Last, *CellSecInspector* derives the end state according to the start state, condition, and actions: after stopping T3540 and responding, the UE remains connected via the N1 NAS signalling connection, but T3540 is no longer running. In a traditional FSM, it is impractical to infer the start and end states, and two operations in the action may bring unnecessary difficulties in determining which edges shall be included within complicated FSM transitions when there are thousands of connected states.

5.3 *Function Chain Builder*

After extracting scattered SCA nodes, each representing a self-contained state transition, we assemble them into coherent function chains, which complete the transitions for representing the complex mobile network functions specified in 3GPP specifications. Notably, this task is non-trivial due to the challenge outlined in **C2** (Section 4.2), where thousands or more SCA nodes can be developed from thousands of pages from a single 3GPP specification. To construct complete function chains from these scattered SCA nodes, two types of connection strategies are designed:

5.3.1 Type I: Node-Informed Exhaustive Connection. This category uses exhaustive pairwise comparisons between SCA nodes to discover all valid transitions. Three connection methodologies are defined:

(a) Temporal Connection. A temporal connection captures the most basic dependency between SCA nodes. It identifies whether the end state of an SCA node is identical to the start state of another. Formally, let the set of extracted SCA nodes be $N = \{n_1, n_2, \ldots, n_m\}$, where each node n_i is defined as

$$n_i = (\text{start}(n_i), \text{condition}(n_i), \text{action}(n_i), \text{end}(n_i)).$$

Each node $n_i \in N$ is associated with a start state and an end state that $\text{start}(n_i)$, $\text{end}(n_i) \in Q$, where Q denotes the set of all specification states extracted from the 5G and 4G NAS and RRC specifications. The Temporal-Connection relation T is defined as $T = \{(n_i, n_j) \in N \times N \mid \text{end}(n_i) = \text{start}(n_j)\}$. SCA node n_j is a valid temporal successor of n_i if the start state of n_j is identical to the end state of n_i. The formalized algorithm is shown in Algorithm 2 in Appendix C. As shown in Figure 3, the SCA Node 2110 extracted from the 5G NAS specification presents a transition from 5GMM-IDLE mode with suspend indication to 5GMM-CONNECTED mode upon receiving an indication from lower layers that the RRC connection has resumed. Another Node 2055 represents a transition starting exactly in 5GMM-CONNECTED mode, describing the transition from 5GMM-CONNECTED to 5GMM-CONNECTED with RRC inactive. Because the end state of Node 2110 is identical to the start state of Node 2055, they meet the requirement for a valid temporal connection.

(b) Semantic Connection. Temporal Connection requires that two SCA nodes must have exactly the same end state and start state. However, 3GPP specifications are written by different experts. There may be slight differences in textual descriptions for the identical concepts or functions in 3GPP specifications, which we refer to as *textually different but semantically equivalent descriptions*. To connect SCA nodes with such a relation, we introduce Semantic Connection, which identifies if the end state $\text{end}(n_i)$ of Node n_i and the start state $\text{start}(n_j)$ of Node n_j are semantically equivalent or strongly related. If so, Node n_j is allowed to be the successor of Node n_i, even when the textual descriptions of $\text{end}(n_i)$ and $\text{start}(n_j)$ differ. Formally, given two SCA nodes, we check whether their semantics are $sem(\text{end}(n_i),\ \text{start}(n_j)) = \text{Yes}$ to associate the SCA nodes with *textual descriptions different but semantically equivalent* for their end states and start states. In our approach, we leverage *CellSecInspector-Core*, which has been trained with extensive domain knowledge, to compare their semantic meaning. It takes 3 steps: (i) the SCA nodes' start and end states are tokenized; (ii) the transformer layers are used to build contextual embeddings based on tokens; (iii) the embeddings are further compared via cosine similarity and enhanced verification via ICL to distinguish deeper logic relations, such as entailment, contradictions, and overlaps. This relation S is defined as

$$S = \left\{(n_i, n_j) \in N \times N \;\middle|\; sem(\text{end}(n_i),\ \text{start}(n_j)) = \text{Yes}\right\}$$

The formalized algorithm is illustrated in Algorithm 3 in Appendix C. As shown in Figure 3, Node 5263 ends in a state where the UE is in 5GMM-IDLE mode and has initiated the registration procedure due to a change in its radio capability. Node 2107 begins in a state where the UE is attempting to send a REGISTRATION REQUEST message. It proceeds to enter 5GMM-IDLE mode and continues the registration procedure. Although the end and start states are described differently, they both refer to the same specification state, the UE initiating a registration procedure triggered by updated radio capability. This represents a valid semantic connection.

(c) Causal Connection. Beyond the direct matching of end and start states, function chains can be causally constructed through causal relationships, where the states, conditions, or actions of one SCA node directly enable, influence, or trigger the states, conditions, or actions of another. That is, the states, conditions, or actions in n_i serve as a plausible prerequisite or cause for the states, conditions, or actions in n_j, suggesting a likely sequential or dependent relationship. Identifying causal connections between nodes is essential to thoroughly construct complete and coherent function chains. We define this Causal-Connection relation C such that for any pair of SCA nodes (n_i, n_j), a connection exists if $C = \{(n_i, n_j) \in N \times N \mid \ \text{causal}(n_i, n_j) = \text{Yes}\ \}$. The formalized algorithm is shown in Algorithm 4 in Appendix C. *CellSecInspector-Core* is employed to infer such causal relations. It analyzes SCA nodes n_i and n_j and determines whether n_i precedes n_j. For example, if the end state of n_i transitions the UE into a mode that is

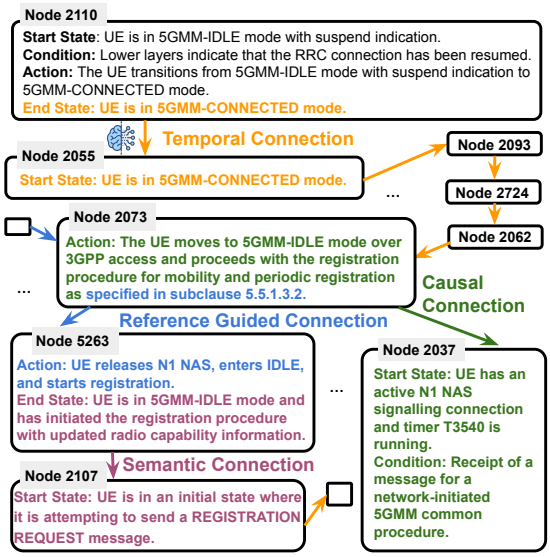


Figure 3: 5G mobility management function chain

required as the start state of n_j, or if the condition in n_i (e.g., radio capability change) leads to a logically subsequent condition in n_j (e.g., registration request trigger), or if the action in n_i (e.g., releasing NAS signaling connection) sets up the environment necessary for the action in n_j (e.g., initiating a new signaling procedure), a causal relationship is inferred.

As shown in Figure 3, Node 2073 models the UE transitioning from 5GMM-CONNECTED to 5GMM-IDLE mode and initiating a registration procedure. As part of this process, the UE establishes an N1 NAS signalling connection, which is necessary for subsequent communication with the network. Node 2037 begins in a state where such a signalling connection is already active and the UE is ready to handle network-initiated 5GMM common procedures. Its condition, receiving a message from the network, relies on the existence of an active signaling context. This requirement is fulfilled by the action in Node 2073, which creates the signaling connection through registration. With this context, Node 2037 can further proceed to receive or respond to the network-initiated procedure. Therefore, the action in Node 2073 is a prerequisite for the state and condition of Node 2037. We identify it as a valid causal connection $(n_{2073}, n_{2037}) \in C$.

5.3.2 Type II: Reference Guided Connection. While the connection strategies in Type I can, in principle, be applied to all SCA nodes, this approach is computationally expensive as discussed in **C3** (see Section 4.2). A single 3GPP specification can yield thousands of nodes, and across the more than 2,000 specifications that define operational 5G and 4G mobile networks [3], exhaustive pairwise validation is infeasible. To reduce the computational cost, we constrain the search space so that each SCA node is only checked with other nodes within the same subclause. However, this restriction risks constructing incomplete function chains. We introduce the Reference Guided Connection to address it. To ensure consistency and maintainability, 3GPP specifications are written in a highly modular style, relying heavily on cross-references to avoid redundancy and to coordinate contributions across multiple working groups. These references are the key to Reference Guided Connection, which works as follows: **1. Reference Detection**: Identify whether an SCA node n_i contains references to specific clauses (e.g., '*as specified in subclause X.Y.Z*'). **2. Space Expansion**: Extract the referenced clause and retrieve all candidate nodes n_j within that clause. **3. Connection Validation**: Apply Type I connections to determine if a valid connection exists between n_i and n_j. The formalized algorithm is shown in Algorithm 5 in Appendix C.

As illustrated in Figure 3, we apply the Reference Guided Connection strategy to link Node 2073 and Node 5263. First, Node 2073 explicitly cites subclause 5.5.1.3.2 in its action field, which specifies the registration procedure triggered by radio capability updates. Based on this reference, all SCA nodes derived from subclause 5.5 are considered candidate targets, including Node 5263. We then apply Type I mechanisms to validate whether a true dependency exists. In this instance, a Causal Connection from Node 2073 to Node 5263 is detected. Specifically, Node 2073 initiates a registration procedure. This behavior corresponds to the more detailed process described in Node 5263, where the registration is triggered by a radio capability change. The action in Node 2073 logically precedes and leads to the action in Node 5263, whose execution realizes the registration behavior initiated in Node 2073. Although their start and end states differ, the procedural flow from Node 2073 to Node 5263 is a causal dependency. Thus, it is a valid Reference Guided Connection.

5.3.3 Complete Function Chain. Applying all proposed connection strategies to analyze the relations among SCA nodes, the *Function Chain Builder* can efficiently and comprehensively construct the function chain. These function chains serve as structured models of complex mobile network services. Figure 3 depicts the constructed function chains that model 5G mobility management.

5.4 SecOracle

SecOracle (Security Oracle) is an adaptive security validation module to assess the security of procedures and functions defined in 3GPP specifications. Current methods, such as exhaustive character-altered testing (i.e., random fuzzing [35]), manually developing mobile network procedure-specific requirements [14, 24], or manually translating FSM models into SMV language [14, 28, 42, 44], are inefficient and unscalable for assessing large-scale function chains (**C2** in Section 4.2).

To address this, we propose a two-step approach: First, we define a set of security properties, which are different from the security requirements in prior works that are specific to each procedure or FSM model [14, 24]. These properties encapsulate the fundamental requirements for entire mobile networks, encompassing both mobile users and network infrastructure operators. They thus can cover a wider range of scenarios and conditions. Derived from security standards (e.g., 3GPP Security Assurance Methodology (SECAM) [2], ITU-T X.805 [1]) and classic security models (e.g., Dolev-Yao [34], Bell-LaPadula [49], and Biba Models [18]), we build 9 core security properties in total (see Appendix E) to cover key security dimensions. The followings are a partial of our defined security properties: `authentication`, ensuring only legitimate users or entities can initiate procedures; `authorization`, enforcing that state transitions, such as NAS transitions, are exclusively triggered

by legitimate, authenticated entities; `service integrity` and `confidentiality`, requiring all communications to be properly secured, with mobile entities verifying message integrity and confidentiality to prevent adversaries from launching attacks such as MitM.

Second, motivated by the fact that cellular network devices, including UEs, base stations, and function modules in Core Network, do message exchanges all the time, 4 fundamental attack methods, including dropping, modifying, injecting, and replaying, denoted as A are applied to each node and node transition. *SecOracle* leverages its reasoning capability to systematically analyze whether function chains still meet all security properties under these 4 attacks. Formally, given $\forall R_i \in$Security Properties, $\forall n_j \in$ C for each function chain, $\forall a_k \in$ A, the security property verification is expressed as, CheckViolation(R_i, n_j, a_k). A candidate violation is flagged and recorded as $C, a_k, n_j \models \neg R_i$ for further analysis. Figure 21 in Appendix C shows how *CellSecInspector-Core* is instructed in this module.

From candidate violations to confirmed vulnerabilities. *SecOracle* outputs candidate violations rather than confirmed vulnerabilities. Both candidate violations and confirmed vulnerabilities refer to *design defects* in 3GPP specifications. A candidate violation indicates that a function chain may violate a security property under an adversarial action and therefore requires further validation. We classify a candidate as a confirmed vulnerability only if it satisfies three criteria: (i) the flagged violation is grounded in the corresponding 3GPP specifications; (ii) the issue reflects a specification-level design weakness rather than only an implementation-specific behavior; and (iii) the issue can lead to a realistic security consequence, such as unauthorized state transition, service disruption, privacy leakage, or inconsistent UE/network state.

Violation example. Consider the function chains shown in Figure 3. For each node, *SecOracle* applies 4 attack methods to assess possible candidate violations against security properties. In Node 2073, the UE sends a REGISTRATION REQUEST to transition from 5GMM-CONNECTED to 5GMM-IDLE mode. As a detected candidate violation shown in Figure 4, under a replay attack, if this REGISTRATION REQUEST is intercepted and replayed, the network may process it again. According to 3GPP 5G NAS specification TS 24.501 subclause 4.4.4.3 [10], the AMF must accept such a message even if it is unauthenticated and lacks integrity protection. The acceptance of an unauthenticated initial REGISTRATION REQUEST is a bootstrapping mechanism in a zero-trust environment. However, as reported in GSMA CVD#0029 [44], the AMF may disconnect an existing legitimate UE connection after receiving an unauthenticated initial REGISTRATION REQUEST, before the requesting entity has been sufficiently authenticated or verified. In this case, the attacker can exploit the fact that an unauthenticated request can trigger security-sensitive connection-state changes. These inconsistencies can lead to billing errors, misrouted downlink traffic, or even session management failures. In severe cases, adversaries can repeatedly force the UE to re-register, thereby disrupting ongoing sessions and achieving a denial-of-service (DoS) attack using forged NAS messages. This is a specification design vulnerability; the violated property is `service integrity`: security-sensitive actions, such as releasing an existing UE connection or replacing an active UE context, should not be triggered solely by an unauthenticated message before sufficient verification.

```
| 2062 | 2073 | Replay | BEGIN_JSON {
  "vulnerability_detected": "Yes",
  "violated_property": [5],
  "explanation": "A Replay attack could exploit the transition from
       5GMM-CONNECTED mode with RRC inactive indication to 5GMM-IDLE mode by
       capturing and retransmitting the REGISTRATION REQUEST message. This
       violates the Replay Attack Protection requirement.",
  "test_case": "An attacker captures the REGISTRATION REQUEST message during
       the transition and retransmits it, causing the AMF to process the same
       message twice, leading to potential registration failures or security
       breaches.",
  "vulnerability_source": "event2073"
} END_JSON |
```

Figure 4: A candidate violation flagged by *SecOracle* for Nodes 2062 and 2073 transition

5.5 VulnTestGenerator

After detecting candidate violations by *SecOracle*, *VulnTestGenerator* generates corresponding testing procedures for validating whether these candidates can be confirmed as vulnerabilities. It aims to narrow the gap between specification-level findings and real-world verifications. As shown in Figure 22 in Appendix C, *VulnTestGenerator* is instructed via ICL to develop structured test cases that specify network states, security context status, UE and core network configurations, operation sequences, and expected outputs, from flagged candidate violations. Those test cases can guide us how to validate and observe the violated security properties. Table 8 in Appendix F presents an example test case for validating the security property `service integrity` violation in Node 2073, which describes a specific behavior in TS 24.501 Subclause 5.3.1.4, where a UE in 5GMM-CONNECTED mode with RRC inactive indication sends a REGISTRATION REQUEST with the NG-RAN-RCU bit set for UE radio capability update and then moves to 5GMM-IDLE.

Validating violations with test cases. Executing the generated test cases on either a controlled testbed (e.g., open5GS [29], srsRAN-[65]) or an operational mobile network allows us to validate the vulnerability and find the root cause. Notably, conducting real-world experiments requires substantial human effort, including building experimental platforms, instrumenting the network/UE to collect experimental data, and analyzing results. The current scope of *VulnTestGenerator* focuses on generating test cases to guide these validation experiments. Executing test cases in an automated manner is a non-trivial activity. We leave it as future work.

6 Evaluation

This section reports the evaluation of *CellSecInspector*. Note that all datasets, scripts, and findings are available on the project website [5]. Implementations and 3GPP specifications used for evaluations are introduced in Appendix D.

6.1 Research Question

To evaluate the performance of *CellSecInspector*, we answer the following research questions.

◇ **RQ1:** Can *CellSecInspector* identify new vulnerabilities in 3GPP specifications?

◇ **RQ2:** How does *CellSecInspector* compare with the state-of-the-art approach in vulnerability discovery effectiveness?

◇ **RQ3:** How effectively does SCA preserve procedural information?

◇ **RQ4:** What's the conversion rate from candidate violations to

confirmed vulnerabilities?
◇ **RQ5:** What is *CellSecInspector*'s runtime performance?

6.2 RQ1: New Vulnerabilities

Among the well-studied TS 38.331, TS 24.501, and TS 24.301, *CellSecInspector* uncovers 5 new vulnerabilities. Beyond these three specifications, we further apply *CellSecInspector* to TS 23.501 (System architecture for the 5G System (5GS)) and TS 24.229 (IP multimedia call control protocol based on Session Initiation Protocol (SIP) and Session Description Protocol (SDP); Stage 3) to demonstrate the extensibility of *CellSecInspector* beyond commonly studied NAS and RRC specifications. Considering that validating each finding requires substantial human effort, *CellSecInspector* analyzes only one selected section from TS 23.501 and TS 24.229, respectively; *CellSecInspector* identifies two additional vulnerabilities from selected sections. Table 1 summarizes all 7 newly identified vulnerabilities with the precise description of the vulnerability, how we conduct the proof-of-concept validation, and the root cause analysis. All 7 findings were validated through controlled experiments using U.S. carrier networks or SDR-based testbeds.

Not every violation flagged by *CellSecInspector* is ultimately reported as a confirmed vulnerability here for two reasons. (1) Some flagged issues cannot be experimentally examined due to the missing support in operational infrastructure. Nevertheless, some flagged issues reflect the same concerns raised by domain experts through 3GPP Change Requests (CRs) before the real-world deployment. For example, since Release 17, TS 23.501 introduces the Unavailability Period mechanism to help the network manage unreachable states of NTN (non-terrestrial networks) UEs. The specification defines how the AMF determines the loss-of-coverage schedule, including the Start of Unavailability Period and the Unavailability Period Duration. However, it does not fully specify the UE-side handling of these configurations, despite their direct impact on paging reception, access stratum operation, and power-saving behavior. This creates a potential design loophole: an NTN UE and the AMF may operate with inconsistent Unavailability Period settings, which can disrupt mobility procedures and trigger transient connectivity failures and unnecessary re-registration attempts. Unfortunately, we did not observe support for the Unavailability Period mechanism in either operational infrastructure or SDR-based platforms. Importantly, after reviewing CRs used by 3GPP to create revised versions of specifications, we found that this out-of-sync Unavailability Period issue was also raised by Samsung in CR 5614 [55]. However, the proposed change does not fully eliminate the inconsistency risk. This alignment with an independently reported CR confirms the value of *CellSecInspector*'s findings, even when full experimental validation is not feasible. Consequently, we submit these potential issues to GSMA and 3GPP rather than reporting them as confirmed vulnerabilities. (2) Confirmed vulnerability reporting requires substantial human effort to investigate identified issues and run validation experiments. Although *CellSecInspector* is extensible to a broad range of 3GPP standards, in this work, we focus on a subset of specifications to demonstrate *CellSecInspector*'s effectiveness in discovering new vulnerabilities.

We next present V1 and V2 in detail below as representative examples, including introductions of vulnerabilities, proof-of-concept validations, and root cause & lessons learned discussions. The remaining V3–V7 vulnerabilities are elaborated in Appendix A following the same structure.

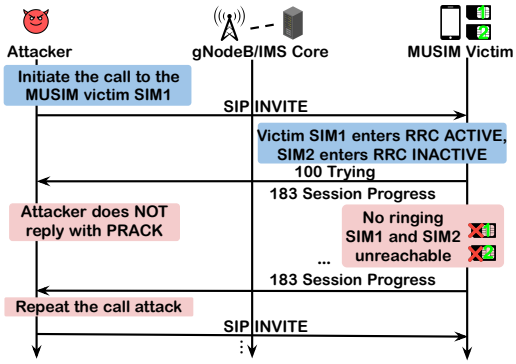


Figure 5: Multi-USIM Problematic Reachability

6.2.1 V1: Multi-USIM problematic reachability. 5G introduces the RRC_INACTIVE state to balance power efficiency with low-latency responsiveness. In this state, a UE remains in 5GMM_CONNECTED mode while its RRC connection is suspended and its session context is stored. This design allows the UE to avoid full reconnection procedures, thereby saving energy and reducing signaling overhead. The mechanism is particularly relevant for multi-USIM (MUSIM) devices, as it allows a SIM in the RRC_INACTIVE state to quickly restore its session to respond to a paging request. This appears to be a reasonable approach, especially for addressing paging collisions [6] by enabling the SIM to quickly receive the message before the request times out.

However, in practice, most modern UEs (e.g., iPhone, Samsung Galaxy, Google Pixel) operate as Dual SIM Dual Standby, sharing a single RF transceiver. Under this hardware constraint, the design is not robust: when the transceiver is fully occupied by SIM1, SIM2 cannot monitor paging occasions or process paging requests. Despite the fact that paging includes a coarse paging cause (e.g., voice, data), the inactive SIM can still miss paging occasions and must rely on network re-paging, creating an exploitable reachability gap.
◇ Proof-of-concept Attack. The victim is a MUSIM user with two SIMs on a commodity off-the-shelf (COTS) phone. The attacker aims to make both SIM subscriptions of the victim persistently unreachable. It assumes that at least one of the victim's SIM numbers can be obtained from publicly available sources, such as social networking platforms, enterprise contact directories, or leaked customer databases. With this minimal knowledge, the attacker can launch a stealthy DoS attack against both SIMs.

Table 2: Multi-USIM DoS attack results

Phone	SIM1	SIM2	Ringing?
Samsung S24	OP-I	OP-II	x
Samsung S24	OP-I	OP-I	x
Samsung S24	OP-I	OP-III	x
Samsung S21	OP-I	OP-II	x
Samsung S21	OP-I	OP-I	x
Samsung S21	OP-I	OP-III	x

As illustrated in Figure 5, the attack proceeds as follows. The attacker first initiates a call to SIM1, which establishes an active

Table 1: Summary of new vulnerabilities

Spec#	Vulnerability	Description	PoC Validation		Root cause
			Validation Setup	Experimental Result	
TS 24.501 v16.8.0 – v20.0.0†	V1: Multi-USIM Problematic Reachability	For MUSIM devices, active traffic on one SIM can prevent the other SIM from receiving paging, which an attacker can exploit to disrupt reachability	U.S. carrier networks; Samsung S24, Samsung S21	Repeated attacker calls to SIM1 prevented SIM2 from ringing across tested carrier combinations	Specification and implementation gap for shared-transceiver MUSIM devices, where reliable paging monitoring for the inactive SIM is not guaranteed
	V2: Emergency Session Teardown via Forged Signaling to Victim	Unprotected emergency REGISTRATION and SIP signaling enable MitM injection that terminates emergency sessions	srsRAN/srsEPC with Linphone IMS with a controlled emergency session setup	Forged RRC RELEASE or SIP CANCEL terminated the victim's emergency session	A threefold design weakness in emergency services: unprotected emergency NAS messages, acceptance of unauthenticated RRC RELEASE, and unprotected SIP signaling
TS 23.501 v16.8.0 – v20.2.0†	V3: Vulnerable DNS-Based PLMN Discovery over Non-3GPP Access	In non-3GPP access, an attacker can use forged DNS responses to redirect a UE to malicious infrastructure before any security context is established	Controlled Wi-Fi network; Samsung S24, Moto G Stylus 5G	DNS poisoning blocked legitimate N3IWF/ePDG access and non-3GPP registration	Unauthenticated DNS-based PLMN discovery over untrusted pre-attachment links enables network misdirection attacks
TS 24.229 v16.8.0 – v20.0.0†	V4: Privacy Exposure via Over-Disclosed VoIMS Signaling Headers	Over-disclosed SIP and SDP headers expose device, user identity, and IMS infrastructure information during VoIMS signaling	U.S. carrier networks; Samsung S21, Pixel 5	SIP traces exposed private information	The text-based SIP signaling design largely increases the risk of transmitting headers with private information
TS 24.301 v16.8.0 – v20.0.0†	V5: Exploitable CS-Fallback Initiation	An attacker sniffs an unprotected extended service request and injects a forged SERVICE REJECT (with EMM cause 39 and T3442). It can trap the victim in a long throttling timer	srsRAN/open5GS SDR testbed; Samsung S24, Moto G Stylus 5G	Devices accepted forged SERVICE REJECT and stopped CSFB call setup until T3442 expired	Specifications allow SERVICE REJECT to be processed without ciphering or integrity protection as a fail-safe mechanism, enabling attackers to inject unauthenticated rejection messages
TS 38.331 v17.0.0 – v19.3.0†	V6: Access Trap via Access Barring Factor	Rogue base station advertises restrictive barringFactor in SIB1/SIB2, causing UEs to suppress MO access attempts and refrain from initiating RRC connection establishment	RF-shielded srsRAN, srsEPC testbed; Samsung S24, Moto G Stylus 5G	Devices self-blocked access after receiving SIB2 with ac-BarringFactor set to 0	UAC policies are broadcast via unauthenticated SIB messages before security is established, creating a pre-authentication trust gap
	V7: Rogue Base Station RRC RESUME Blackhole	By exploiting unauthenticated RRC RESUME procedure, a rogue gNB traps the victim UE in a continuous loop of connection failures	RF-shielded gNB setup; Samsung S21, Moto G Stylus 5G	Devices were attracted to the rogue gNB and repeatedly failed RRC RESUME	Cell selection and reselection occur at the RRC layer before a protected AS channel is established, and the mechanism does not account for resume failures

† The latest 3GPP specification version available at the time of submission.

RRC connection for SIM1 and forces SIM2 into RRC INACTIVE. To avoid alerting the victim, the attacker can leverage the ghost call attack [53, 68], preventing the call from ringing while keeping SIM1 trapped in the RRC ACTIVE state during the entire attack period. Note that, under the ghost call attack, SIM1 is also unable to receive any other incoming calls. Consequently, SIM2 remains stuck in RRC INACTIVE. All incoming paging requests for SIM2 will be ignored. The device neither rings nor displays an incoming call notification. As a result, the victim UE becomes persistently unreachable through both SIM1 and SIM2, despite having two independent subscriptions.

We validate this attack on a Samsung Galaxy S24 and a Samsung Galaxy S21. We choose these devices because both provide two physical SIM slots, which simplifies repeated experiments and SIM swapping without triggering the additional security restrictions often associated with moving eSIM profiles across devices. During the interval in which the attacker withholds PRACK, we use another phone to call SIM2. As summarized in Table 2, neither device produces a ringing notification for SIM2 on any tested carrier. Notably, this attack does not strictly require a ghost call. The attacker can also keep repeatedly calling SIM1 or maintain an ongoing connection with SIM1 to occupy the shared RF resources. Although this variant is more likely to attract the victim's attention, SIM2 still does not receive incoming call notifications during the attack period.

◇ **Root Cause and lessons learned.** The root cause is a mismatch between specification assumptions and widely deployed MUSIM hardware. The 5G NAS specification [10] implicitly assumes device capabilities that can maintain simultaneous accessibility for multiple SIMs, whereas most modern MUSIM devices share a single RF module. The specifications do not mandate mechanisms that ensure all MUSIM devices, in particular, devices with a shared transceiver, can reliably monitor and process Paging Requests. This gap enables an attacker to exploit the MUSIM paging mechanism to attack MUSIM reachability. Mitigating this issue likely requires combined core-network and device-side enhancements in future 3GPP releases. For example, MUSIM devices could implement adaptive internal scheduling for ongoing traffic/calls and bring SIMs in RRC INACTIVE to ACTIVE for paging monitoring, rather than ignoring paging on the inactive SIM.

6.2.2 V2: Emergency Session Teardown via Forged Signaling to Victim. To ensure emergency services remain accessible to all users, 5G NAS specification TS 24.501 [10] permits UEs without valid credentials to connect. If an unregistered UE attempts to make an emergency call (e.g., 911 call), it initiates a NAS REGISTRATION REQUEST with the type set to emergency registration. Since this message may be transmitted without encryption or an established NAS security context, it opens a loophole that adversaries can monitor and use a rogue base station to send fabricated responses to the UE, enabling a man-in-the-middle (MiTM) attack. The exposure extends to the IMS layer: TS 24.229 permits emergency SIP signaling without authentication or IPsec protection, leaving critical emergency call-control messages vulnerable to interception and manipulation [23, 38, 40, 41].

Is it the same vulnerability in [39]? This vulnerability is fundamentally different from previously reported vulnerabilities [39], which exploit the fact that the infrastructure accepts unverifiable emergency session requests over the IP-CAN session and suffers from improper cross-layer security binding. These are infrastructure-side logic loopholes, and operators could mitigate them by temporarily allowing duplicate emergency IP-CAN sessions. Our V2 targets a loophole in the standard-permitted emergency entry path. It enables a MiTM attack at the radio and emergency signaling layers directly targeting the UE; therefore, V2 cannot be noticed by the carrier networks.

◇ **Proof-of-concept Attacks.** Two proof-of-concept attacks are used to validate this vulnerability.

Attack 1: Forcing emergency radio session release. This attack exploits the NAS registration and RRC release procedures. When an unregistered UE initiates an emergency call, it sends a NAS REGISTRATION REQUEST with the type set to emergency. Because the UE does not have a valid security context, this message is

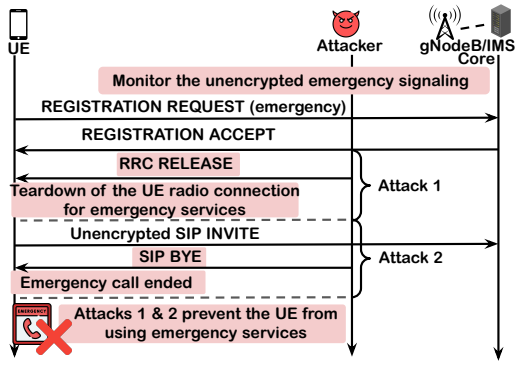


Figure 6: Emergency Session Teardown via Forged Signaling to Victim

```
00:39:55.051] [rrc] [debug] RRC Release received
00:39:55.052] [nas] [info] UE switches to state [5U2-NOT-UPDATED]
00:39:55.052] [nas] [info] UE switches to state [MM-DEREGISTERED/PS]
00:39:55.052] [nas] [debug] Initial registration required due to [DUE-TO-SERVI
00:39:55.052] [nas] [info] UE switches to state [CM-IDLE]
```

Figure 7: The forged RRC Release message traps the victim UE by disconnecting it from the base station

transmitted unprotected. Therefore, an attacker operating a rogue base station can capture both REGISTRATION REQUEST and the REGISTRATION ACCEPT from the legitimate network. Next, the attacker can send a forged RRC RELEASE to the UE, forcing it to tear down the ongoing radio connection for the upper emergency services.

Due to ethical considerations, we validate this attack in a fully controlled environment rather than interacting with the operational networks or real 911 call center. Specifically, we deploy a serving network using srsRAN and srsEPC [65], together with an IMS service based on the Linphone VoIP server [16]. Additionally, we set up another set of srsRAN and srsEPC as the attacker on the same server. We use UERANSIM [36] as the victim UE. After the UE receives the Registration Accept, the attacker keeps sending a forged RRC RELEASE to the victim UE. In wild settings, the attacker needs to do an RF scan, detect SSB/PBCH, and read MIB/SIB1 once synchronized for sending the forged RRC message on the same downlink carrier the victim is using. Figure 7 shows the victim attempts to reconnect to continue the emergency service. The attacker can continue sending the forged RRC RELEASE to prevent the victim from getting the radio connection.

Attack 2: Ghost emergency call cancellation. It targets the unprotected SIP signaling for emergency voice services. In this attack, the attacker leverages the fact that the 3GPP specification permits emergency services without authentication or encryption. The attacker intercepts the unencrypted SIP INVITE and spoofs a forged SIP CANCEL message, deceiving the emergency UE that the emergency call session has been legitimately terminated. We validate this attack using the aforementioned in-house setting for ethical considerations. After observing the unencrypted SIP INVITE, the attacker can send a forged SIP CANCEL to the victim to cancel the call. Our result confirms that the victim's emergency call is canceled after receiving a 'ghost' cancellation signaling message.

◇ **Root Cause and Lesson Learned.** These attacks stem from the lack of protection in the REGISTRATION REQUEST and the design trade-offs in the emergency service procedures to guarantee universal accessibility. Together, they expose unprotected NAS and SIP signaling to attackers, making mobile users vulnerable to emergency service DoS attacks. The root cause is therefore threefold: (1) the omission of mandatory encryption and authentication for emergency NAS messages, (2) the acceptance of RRC RELEASE without protections, and (3) the allowance of unprotected SIP signaling for emergency voice services. Future 3GPP specification releases must revisit the emergency service procedures and incorporate mitigations that preserve accessibility without sacrificing security. For instance, lightweight integrity-protection mechanisms could be mandated for emergency signaling, ensuring messages cannot be silently modified. Different from [39], our proposed attacks target emergency UEs; therefore, they are harder to be noticed by carriers.

6.3 RQ2: Vulnerability Discovery Comparison

Beyond discovering new vulnerabilities, it is important to understand how *CellSecInspector* performs relative to prior art. Accordingly, we compare *CellSecInspector* with the state-of-the-art approach in terms of vulnerability discovery effectiveness, which is intended to assess whether *CellSecInspector* can have a better coverage of security flaw identification in 3GPP specifications. As discussed in Section 3, some existing approaches [24, 25, 66] target implementation-level issues instead of detecting design flaws in 3GPP specifications. *CellularLint* [61] aims to detect sentence-level semantic conflicts in 3GPP specifications rather than finding the design vulnerabilities. To the best of our knowledge, *Hermes* [14] is the only comparable system that can also discover vulnerabilities from standards. To enable a fair comparison, we collect and develop a benchmark set of known vulnerabilities [14, 15, 19, 22, 26, 27, 42–44, 46, 48, 50, 51, 54, 58, 64, 67, 69] that are design defects from same specifications (TS 38.331, TS 24.501, TS 24.301) used by *Hermes*.

As shown in Table 3, *Hermes* can detect 22 of all 36 known vulnerabilities in 5G and 4G NAS and RRC specifications. After manually verifying the candidate violations flagged by *CellSecInspector*, we confirm that flagged violations can cover all of these KNOWN vulnerabilities. The vulnerability discovery results show that *CellSecInspector* is capable of effectively modeling mobile network procedures, conducting security analysis, and identifying security threats in 3GPP specifications. Combined with the results in RQ1, *CellSecInspector* not only has better vulnerability discovery coverage, but also can discover new vulnerabilities.

Why are there more known vulnerabilities than new vulnerabilities? The 36 known vulnerabilities form a cumulative benchmark collected from the aforementioned prior studies over several years, whereas the 7 new vulnerabilities include only previously unreported, experimentally validated design flaws identified in this work. As clarified in RQ1, we also exclude duplicate, implementation-dependent, and unconfirmed candidates from the new-Reference Guided Connection count.

6.4 RQ3: How effectively does SCA preserve procedural information?

SCA nodes are the core modeling representation of *CellSecInspector*. As introduced in Section 5.2, each SCA node represents one self-contained transition using four fields. The vulnerability-discovery results in RQ1 and RQ2 demonstrate the effectiveness of *CellSecInspector*, but do not explain whether SCA preserves the procedural

Table 3: Comparison of KNOWN vulnerabilities detected by Hermes and *CellSecInspector*

ID	Attack	H*	C**
1	Downgrade to non-LTE network services [64]	✓	✓
2	Denying all network services [64]	✓	✓
3	Denying selected service [64]	×	✓
4	Signaling DoS [15, 46, 50, 51]	✓	✓
5	S-TMSI catching [48]	✓	✓
6	IMSI catching[67]	✓	✓
7	EMM Information [58]	✓	✓
8	Impersonation attack [26]	×	✓
9	Synchronization Failure attack [69]	×	✓
10	Malformed Identity Request [54]	×	✓
11	Neutralizing TMSI refreshment [44]	×	✓
12	NAS Counter Reset [44]	✓	✓
13	Uplink NAS Counter Desynchronization [44]	✓	✓
14	Exposing NAS Sequence Number [44]	✓	✓
15	Cutting off the Device [44]	×	✓
16	Exposure of SQN [19]	✓	✓
17	5G AKA DoS Attack[22]	✓	✓
18	SUCI catching [27]	×	✓
19	IMSI cracking [43]	×	✓
20	NAS COUNT update attack [14]	✓	✓
21	Deletion of allowed CAG list [14]	✓	✓
22	Downgrade using ATTACH/REGISTRATION REJECT [64]	✓	✓
23	AUTHENTICATION REJECT attack [69]	✓	✓
24	DETACH/DEREGISTRATION REQUEST attack [42]	✓	✓
25	SERVICE REJECT attack [64]	✓	✓
26	Denial-of-Service with RRC SETUP REQUEST attack [44]	×	✓
27	Installing Null Cipher and Null Integrity [44]	✓	✓
28	Lullaby Attack [44]	✓	✓
29	Incarceration with RRC REJECT/RELEASE [44]	✓	✓
30	Measurement report [64]	×	✓
31	RLF report [64]	✓	✓
32	Blind DoS attack [48]	×	✓
33	AKA bypass [48]	×	✓
34	Paging channel hijacking [42]	×	✓
35	Energy Depletion with RRC SETUP [14]	✓	✓
36	V2X Message Spoofing over PC5 [63]	×	✓
Detected Ratio		**22/36**	**36/36**

H*: *Hermes*, C**: *CellSecInspector*

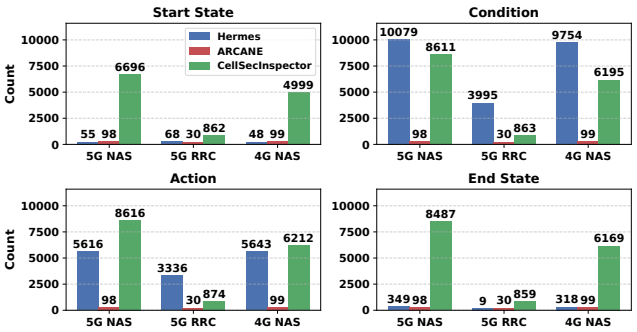


Figure 8: Quantity Comparison of SCA nodes and FSMs

Table 4: Comparison between an SCA node from *CellSecInspector* and an FSM transition from *Hermes*

Element	*CellSecInspector* (SCA Node)	*Hermes* (FSM Transition)
Start State	UE is likely in 5GMM-REGISTERED.	<start_state> A UE enters the state 5GMM-SERVICE-REQUEST-INITIATED </start_state>
Condition	After the UE has started the service request procedure AND is waiting for a response from the network.	<condition> it has started the service request procedure </condition> <condition> is waiting for a response from the network </condition>
Action	UE enters the 5GMM-SERVICE-REQUEST-INITIATED state.	N/A
End State	UE is in the 5GMM-SERVICE-REQUEST-INITIATED state, waiting for a response from the network.	N/A

Table 5: SCA completeness comparison of Hermes, ARCANE, and *CellSecInspector*

Spec#	Method	0 fields	1 field	2 fields	3 fields	4 fields
24.501	Hermes	1122 (20.1%)	1055 (19.0%)	2177 (39.1%)	207 (3.7%)	5 (0.1%)
	CellSec	1778 (17.1%)	36 (0.3%)	132 (1.3%)	1920 (18.4%)	6549 (62.9%)
	ARCANE	0 (0%)	0 (0%)	0 (0%)	0 (0%)	98 (100%)
38.331	Hermes	2948 (46.2%)	2675 (41.9%)	753 (11.8%)	1 (0.0%)	0 (0.0%)
	CellSec	321 (26.9%)	11 (0.9%)	1 (0.1%)	7 (0.6%)	855 (71.5%)
	ARCANE	0 (0%)	0 (0%)	0 (0%)	0 (0%)	30 (100%)
24.301	Hermes	3026 (50.6%)	463 (7.7%)	2244 (37.5%)	237 (4.0%)	6 (0.1%)
	CellSec	1771 (22.2%)	30 (0.4%)	13 (0.2%)	1207 (15.1%)	4974 (62.2%)
	ARCANE	0 (0%)	0 (0%)	0 (0%)	0 (0%)	99 (100%)

information (i.e., states, conditions, and actions) required for downstream function-chain construction and security analysis in *Function Chain Builder* and *SecOracle*. Because FSMs are the dominant representation used by prior specification-analysis systems, we next compare SCA nodes with the conventional FSMs synthesized by prior works. The goal of this evaluation is to isolate the contribution of SCA and examine whether SCA can better preserve the procedural information needed for downstream tasks.

Hermes [14] is the closest reference system, which extracts protocol models from 3GPP specifications for security analysis. *ARCANE* [66] is also included, while its FSMs are designed for implementation-level model-based fuzzing rather than specification-level design-flaw analysis. All three approaches are applied to the same 5G/4G NAS and RRC specifications. Because conventional FSMs do not expose conditions and actions as explicit fields, we normalize the representations into the same four-field structure: start state, condition, action, and end state. For *Hermes*, condition and action information is derived from its parsed FSM transitions. For *ARCANE*, we adopt its manual approach to map its message-level transitions and information-element constraints into the same structure. We then showcase the evaluation using three metrics: quantity, completeness, and accuracy.

Quantity. Figure 8 shows the number of extracted fields across the three specifications. There are two key observations. (1) SCA captures substantially more start and end states than the two FSM-based approaches, indicating more derived mobile network functions from 3GPP specifications. Table 4 compares an SCA node derived by *CellSecInspector* and an FSM transition annotated by *Hermes* for a sentence '*A UE enters the state 5GMM-SERVICE-REQUEST-INITIATED after it has started the service request procedure and is waiting for a response from the network.*' from TS 24.501. While the sentence does not explicitly specify the start state, *CellSecInspector* reasonably infers that the UE is likely in the 5GMM-REGISTERED state and correctly recognizes two sequential conditions, the action, and the end state. (2) *Hermes* reports a larger number of extracted conditions and actions in 5G and 4G NAS and RRC specifications. However, it is primarily due to splitting closely connected conditions and actions within a single sentence into multiple independent entries, as shown in Table 4. This fragmentation inflates raw counts of extracted elements but fails to preserve the internal structure and logical interdependencies among conditions and actions.

Completeness. We evaluate the completeness of state transitions. A state transition is defined to be complete if all 4 corresponding start, condition, action, and end fields can be derived and are not placeholders (e.g., "Not specified" or "Not explicitly defined"). More

Table 6: Expert evaluation results

(a) Expert judgment of SCA nodes and FSMs

Method	Total	Accuracy	$\{a, b, c, d\}$	Raw Agr.	κ
CellSecInspector	3147	98.92%	{3113, 13, 8, 13}	99.33%	0.80
Hermes	3634	25.23%	{917, 32, 28, 2657}	98.35%	0.96
ARCANE	162	0.00%	{0, 0, 0, 162}	100.00%	N/A

(b) Expert validation of SCA node extraction across 4 models

Model	Expert1	Expert2	Avg. Acc.	Cohen's κ
DeepSeek-V3.2	1.00	1.00	1.00	1.00
Qwen-Plus	1.00	1.00	1.00	1.00
Qwen3-4B	0.90	0.90	0.90	0.92
DeepSeek-R1-8B	0.90	1.00	0.95	0.94

complete fields indicate that more information is likely derived from 3GPP specifications.

As shown in Table 5, it is observed that a large portion of SCA transitions contain three or four valid fields, and 62.2%–71.5% of the extracted transitions are fully specified across the three specifications. *Hermes* extracts many partial transitions, while *ARCANE* produces a much smaller set of structurally complete message-level transitions because its representation is intentionally tailored to fuzzing. These results show that SCA provides a useful balance between broad procedural coverage and preservation of the information required for downstream security analysis.

Accuracy. We evaluate whether an SCA node or an FSM is accurate based on two criteria: (1) evidence-grounded correctness, whether each extracted representation accurately reflects the protocol behavior described in the specification and is supported by explicit textual evidence, and (2) semantic consistency, whether the extracted representation preserves the intended meaning of the mobile network function behavior described in 3GPP specifications. Because this judgment requires domain expertise in cellular network standards, it cannot be reliably outsourced to crowdworkers. We thus ask two domain experts to independently judge whether an SCA node/an FSM is accurate. Formally, each SCA node and FSM is labeled as $v_i \in \{Pass, Fail\}$. We define that an SCA node or FSM is accurate only if both experts label it as Pass. Based on the annotations from two experts, we construct a contingency table $\{a, b, c, d\}$, where a denotes Pass/Pass, b denotes Pass/Fail, c denotes Fail/Pass, and d denotes Fail/Fail. Due to the large amount of SCA nodes and FSMs, two experts are asked to assess the SCA nodes and FSMs extracted from the selected sections, including Clause 4 of 4G NAS specification, Clause 4 of 5G NAS specification, Clauses 4 and 5 of 5G RRC specification. We report the acceptance rate (a/N), raw agreement $((a + d)/N)$, and Cohen's κ in Table 6 (a).

The results show that the SCA representation more consistently preserves 3GPP specification-grounded and semantically coherent protocol behaviors. The main advantage is not the use of a different label format, but the explicit binding of the start state, triggering condition, prescribed action, and end state within one transition. By comparison, FSMs from *Hermes* and *ARCANE* may be suitable for their own downstream tasks (i.e., parsing or fuzzing). However, both experts agree that FSMs from *Hermes* and *ARCANE* struggle to model the mobile network functions.

We also assess whether the accuracy of extracted SCA nodes can be affected if *CellSecInspector-Core* is built on different vanilla models. Four models, **Qwen-Plus-2025-12-01**, **Qwen3-4B-Instruct-2507**, **DeepSeek-V3.2 Release 2025/12/01**, and **DeepSeek-R1-Distill-Llama-8B**, are used to adapt cellular domain knowledge and extract SCA nodes. As shown in Table 6(b), all four models achieve high sampled accuracy, while the larger models perform best. This result indicates that the SCA formulation is transferable across different base models and larger models are preferable choices since the capability still affects extraction quality.

6.5 RQ4: Candidate Violation to Confirmed Vulnerability Conversion Rate

As introduced in Sections 5.4 and 6.2, not every candidate violation can be assessed and validated as a confirmed vulnerability. We therefore adopt strict, conservative criteria to measure the conversion rate from candidate violations to confirmed vulnerabilities: a candidate violation identified by *CellSecInspector* is classified as a confirmed vulnerability only if it satisfies the following criteria: (i) it is attributable to a standard-defined procedure; (ii) it is non-duplicate; (iii) it is not solely implementation-dependent; and (iv) it is experimentally validated when the required testing support is available. Candidates that are duplicates, implementation-dependent, or currently unverifiable due to missing operational or SDR support are not counted as confirmed vulnerabilities. Under these strict conservative criterion, *CellSecInspector* reports 90 candidate violations in TS 24.501, of which 18 are confirmed as valid design-defect instances; 62 candidates in TS 24.301, of which 15 are confirmed; 161 candidates in TS 38.331, of which 12 are confirmed; 36 candidates in the analyzed section of TS 23.501, of which 1 is confirmed; and 8 candidates in the analyzed section of TS 24.229, of which 1 is confirmed. Known vulnerabilities 22–25 in Table 3 occur independently in both 5G and 4G specifications and are therefore counted as separate confirmed instances.

The relatively low conversion rate is mainly caused by repeated reports of the same root cause, implementation-dependent cases, and candidates that cannot yet be validated due to limited infrastructure support. For example, identical RRC messages reused across multiple TS 38.331 procedures can generate several candidate violations corresponding to one underlying weakness. Thus, these conversion rates should not be interpreted as the accuracy of the final confirmed vulnerability set. Instead, they are lower-bound rates over all candidate violations produced by *CellSecInspector*, which narrows a large specification analysis space to a smaller set of security-relevant candidates for expert review and validation.

6.6 RQ5: Runtime Performance

We measure the end-to-end processing time of key components, including *SCA Representation Extractor*, *Function Chain Builder*, *SecOracle*, and *VulnTestGenerator*. The reported runtime is measured using DeepSeek-V3.2 [32] via its public API through a workstation running Ubuntu 24.04 LTS with an NVIDIA RTX 4090 GPU. For the full specifications TS 24.501 (5G NAS), TS 24.301 (4G NAS), and TS 38.331 (5G RRC), and the selected sections of TS 23.501 (5G system architecture) and TS 24.229 (IMS signaling), the *SCA Representation Extractor* derives 10,415, 7,995, 1,195, 1,253, and 320 SCA nodes in 17.36, 13.33, 1.99, 2.09, and 0.53 hours, respectively. The *Function Chain Builder* is the primary runtime bottleneck. It takes approximately 60 days, 30 days, 4 days, 0.44 days, and 0.03 days to construct 11,273, 6,943, 222, 83, and 6 function chains, respectively, with an average of 5.46 seconds per analyzed SCA pair.

Approximately 99% of this time is spent identifying semantic and causal connections. Reference Guided Connection substantially reduces the candidate search space. *Without it, exhaustive all-pair analysis becomes infeasible in our experimental setting. For example, on TS 24.501, our design reduces the estimated runtime from approximately 18.78 years (*$10,415^2 \times \frac{5.46\ sec}{3600\times24\times365} = 18.78$ *years) to 60 days.* The *SecOracle* and *VulnTestGenerator* together complete vulnerability analysis and test case generation in 17.54, 10.80, 0.35, 2.10, and 0.54 hours for the five specifications, respectively.

Scalability limit. Although *CellSecInspector* improves the scalability of specification analysis, corpus-wide 3GPP analysis remains computationally challenging. Directly linking all SCA nodes across hundreds of specifications would create a prohibitively large pairwise search space, and the resulting semantic and causal checks would dominate runtime. Therefore, corpus-wide analysis requires further efforts, such as our reference-guided design that reduces the process from years to days, to achieve with limited computational resources. A practical deployment should prioritize security-critical specifications, such as the TS 23/24 control-plane families; *CellSecInspector* should be executed in parallel to boost the runtime performance. Overall, the current results demonstrate the feasibility of automated security analysis for large, security-critical specifications while identifying scalability as an important direction for future work.

7 Discussion and Limitations

Can the defined security properties and four attacks guarantee discovery of all vulnerabilities? *CellSecInspector* aims to reduce reliance on slow, manual expert review by enabling systematic and scalable detection of standard-level defects in the extracted function chains. It does not guarantee the discovery of all vulnerabilities for two main reasons. First, security verification is inherently scoped. Any finite set of security properties may miss emerging or unmodeled failure modes. We can only claim security with respect to the predefined properties and assumptions of the threat model [14, 24]. Second, while not all SCA nodes correspond to explicit message exchanges, some nodes represent event or condition-driven transitions such as local policy decisions. In this case, *SecOracle* still applies the four attacks to such nodes by exploiting their triggered events or conditions. If an SCA node is purely deterministic without any inputs, such as entering one state after another state, this transition is non-attackable under our settings. Notably, the majority of reported vulnerabilities in RRC and NAS layers included in Table 3 stem from message exchanges and thus fall within the scope of our attack model.

Out-of-scope vulnerabilities. *CellSecInspector* currently targets standard design flaws that can be exposed through adversarial manipulation of message or event flows, including dropping, modifying, injecting, and replaying messages. This scope covers availability, authentication, authorization, integrity, confidentiality, and state-consistency issues in control-plane procedures. We admit that several important vulnerability classes are outside the current model. For example, *CellSecInspector* does not directly analyze cryptographic weaknesses, side channels, physical-layer or social-engineering attacks, and vendor deviations absent from specifications, and may miss deployment-specific vulnerabilities involving carrier policies or undocumented hardware.

Specification evolution and cross-release analysis.

3GPP specifications continuously evolve across releases. Previously identified security issues may persist, be partially mitigated, or be eliminated by later revisions. These ongoing efforts are essential for strengthening the security of cellular systems. At the same time, the scale and complexity of the standards make it challenging to identify and eliminate every security-relevant design weakness through traditional methods. Particularly, we examined the specification text corresponding to our newly reported vulnerabilities in the latest available versions at the time of submission and found that all 7 vulnerabilities remain present, as shown in Table 1. This observation motivates the automated cross-release analysis. In future work, we plan to extend *CellSecInspector* to compare SCA nodes and function chains across versions, identify security-relevant changes, and determine whether previously identified weaknesses are mitigated, persist, or are reintroduced as specifications evolve.

Ethical Consideration and Responsible Disclosure. Our research follows established ethical standards for security research. Identified vulnerabilities in cellular specifications are handled through responsible disclosure. We work with stakeholders, including standardization organizations, operators, and device manufacturers, to report design defects. Their acknowledgments will be posted on our project website. Our objective is not to expose flaws for exploitation, but rather to advance an efficient methodology for enabling 3GPP, operators, and vendors to rigorously review specifications prior to implementation and public deployment of services.

8 Conclusion

Security assurance in cellular networks relies on rigorous validation of 3GPP specifications, yet automation remains challenging due to complex semantics, cross-clause dependencies, and evolving standards. In this work, we present *CellSecInspector*, an automated framework for security assessment of 3GPP specifications. Through five coupled components, *CellSecInspector* transforms cellular network standards into structured SCA nodes, connects scattered procedures into function chains, checks them against security properties under four adversarial scenarios, and generates test cases for validation. Applying *CellSecInspector* to selected 5G and 4G specifications, we identify 43 vulnerabilities, including 36 known vulnerabilities and 7 new vulnerabilities. These results show that *CellSecInspector* can narrow large specifications to a small set of security-relevant candidates for expert validation. *CellSecInspector* can also serve as a specification-time assistant, helping 3GPP contributors detect design flaws before they propagate into implementations. Future work will explore cross-specification and cross-release analysis, broader threat models, improved candidate prioritization, and automated test execution.

9 Acknowledgments

This work was supported in part by the National Science Foundation (NSF) under Grants CNS-2321416, CNS-2246050, and NAIRR-240456. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the NSF. We also thank the shepherd and reviewers for their help in improving this material.

References

[1] 2003. *Security architecture for systems providing end-to-end communications*. Technical Report X.805. ITU-T.

[2] 2020. *Security Assurance Methodology (SECAM) for 3GPP network products*. Technical Report TR 33.916, v16.0.0. ETSI / 3GPP. Technical report, Release 16. https://www.etsi.org/deliver/etsi_tr/133900_133999/133916/16.00.00_60/tr_133916v160000p.pdf

[3] 2025. 3GPP Specifications. https://www.3gpp.org/specifications. Accessed: 2025-08-31.

[4] 2025. Jetstream2 launches large language model (LLM) inference service. Online; Indiana University Jetstream2 News & Events. Accessed on 28 August 2025. https://jetstream-cloud.org/news-events/news/4-11-25_llm-inference-service.html

[5] 2026. CellSecInspector: Safeguarding Cellular Networks via Automated Security Analysis on Specifications. Accessed: 2026. https://github.com/Chelsiehi/CellSecInspector-MobiCom26

[6] 3GPP. 2022. Support for Multi-SIM Devices for LTE/NR. Online; 3GPP Technologies. Accessed on 28 August 2025. https://www.3gpp.org/technologies/support-for-multi-sim-devices-for-lte-nr

[7] 3GPP. 2025. Specifications. Online. [Online]. Available: https://www.3gpp.org/ftp/Specs.

[8] 3rd Generation Partnership Project (3GPP) 2020. *Evolved Universal Terrestrial Radio Access (E-UTRA); User Equipment (UE) procedures in idle mode (3GPP TS 36.304 version 16.2.0)*. 3rd Generation Partnership Project (3GPP).

[9] 3rd Generation Partnership Project (3GPP) 2021. *Universal Mobile Telecommunications System (UMTS); LTE; 5G; Non-Access-Stratum (NAS) protocol for Evolved Packet System (EPS); Stage 3 (3GPP TS 24.301 version 16.8.0 Release 16)*. 3rd Generation Partnership Project (3GPP).

[10] 3rd Generation Partnership Project (3GPP) 2022. *5G; Non-Access-Stratum (NAS) protocol for 5G System (5GS); Stage 3 (3GPP TS 24.501 version 16.8.0 Release 16)*. 3rd Generation Partnership Project (3GPP).

[11] 3rd Generation Partnership Project (3GPP) 2022. *TS 38.331: NR; Radio Resource Control (RRC); Protocol specification (3GPP TS 38.331 version 17.0.0 Release 17)*. 3rd Generation Partnership Project (3GPP).

[12] 3rd Generation Partnership Project (3GPP) 2025. *System architecture for the 5G System (5GS) (3GPP TS 23.501 version 20.0.0 Release 20)*. 3rd Generation Partnership Project (3GPP).

[13] 3rd Generation Partnership Project (3GPP) 2026. *Digital cellular telecommunications system (Phase 2+); Universal Mobile Telecommunications System (UMTS); LTE; IP Multimedia Call Control Protocol based on Session Initiation Protocol (SIP) and Session Description Protocol (SDP) (3GPP TS 24.229 version 19.5.0 Release 19)*. 3rd Generation Partnership Project (3GPP).

[14] Abdullah Al Ishtiaq, Sarkar Snigdha Sarathi Das, Syed Md Mukit Rashid, Ali Ranjbar, Kai Tu, Tianwei Wu, Zhezheng Song, Weixuan Wang, Mujtahid Akon, Rui Zhang, et al. 2024. Hermes: Unlocking security analysis of cellular network protocols by synthesizing finite state machines from natural language specifications. In *33rd USENIX Security Symposium (USENIX Security 24)*. 4445–4462.

[15] Ramzi Bassil, Imad H. Elhajj, Ali Chehab, and Ayman Kayssi. 2013. Effects of Signaling Attacks on LTE Networks. In *2013 27th International Conference on Advanced Information Networking and Applications Workshops*. 499–504. doi:10.1109/WAINA.2013.136

[16] Belledonne Communications. [n. d.]. Linphone Open Source VoIP Softphone. https://www.linphone.org/en/.

[17] Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Zhe Fu, et al. 2024. Deepseek llm: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954* (2024).

[18] K. J. Biba. 1977. *Integrity Considerations for Secure Computer Systems* (esd-tr-76-372 ed.). Technical Report. MITRE Corp. 66 pages.

[19] Ravishankar Borgaonkar, Lucca Hirschi, Shinjo Park, and Altaf Shaik. 2018. New privacy threat on 3G, 4G, and upcoming 5G AKA protocols. *Cryptology ePrint Archive* (2018).

[20] Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. 2015. A large annotated corpus for learning natural language inference. arXiv:1508.05326 [cs.CL] https://arxiv.org/abs/1508.05326

[21] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[22] Jin Cao, Maode Ma, Hui Li, Ruhui Ma, Yunqing Sun, Pu Yu, and Lihui Xiong. 2020. A Survey on Security Aspects for 3GPP 5G Networks. *IEEE Communications Surveys & Tutorials* 22, 1 (2020), 170–195. doi:10.1109/COMST.2019.2951818

[23] Min-Yue Chen, Yiwen Hu, Guan-Hua Tu, Chi-Yu Li, Sihan Wang, Jingwen Shi, Tian Xie, Ren-Chieh Hsu, Li Xiao, Chunyi Peng, Zhaowei Tan, and Songwu Lu. 2024. Taming the Insecurity of Cellular Emergency Services (9–1-1): From Vulnerabilities to Secure Designs. *IEEE/ACM Transactions on Networking* 32, 4 (2024), 3076–3091. doi:10.1109/TNET.2024.3379292

[24] Yi Chen, Di Tang, Yepeng Yao, Mingming Zha, XiaoFeng Wang, Xiaozhong Liu, Haixu Tang, and Baoxu Liu. 2023. Sherlock on Specs: Building {LTE} Conformance Tests through Automated Reasoning. In *32nd USENIX Security Symposium (USENIX Security 23)*. 3529–3545.

[25] Yi Chen, Yepeng Yao, XiaoFeng Wang, Dandan Xu, Chang Yue, Xiaozhong Liu, Kai Chen, Haixu Tang, and Baoxu Liu. 2021. Bookworm game: Automatic discovery of lte vulnerabilities through documentation analysis. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1197–1214.

[26] Merlin Chlosta, David Rupprecht, Thorsten Holz, and Christina Pöpper. 2019. LTE security disabled: misconfiguration in commercial networks. In *Proceedings of the 12th conference on security and privacy in wireless and mobile networks*. 261–266.

[27] Merlin Chlosta, David Rupprecht, Christina Pöpper, and Thorsten Holz. 2021. 5G SUCI-Catchers: Still catching them all?. In *Proceedings of the 14th ACM Conference on Security and Privacy in Wireless and Mobile Networks*. 359–364.

[28] Edmund Clarke, Kenneth McMillan, Sérgio Campos, and Vasiliki Hartonas-Garmhausen. 1996. Symbolic model checking. In *International conference on computer aided verification*. Springer, 419–422.

[29] Open5GS Community. [n. d.]. Open5GS: Open Source Implementation of 4G/5G Core Network. https://open5gs.org/. Accessed: 2025-08-31.

[30] Ido Dagan, Oren Glickman, and Bernardo Magnini. 2005. The pascal recognising textual entailment challenge. In *Machine learning challenges workshop*. Springer, 177–190.

[31] DeepSeek. 2025. DeepSeek-R1-Distill-Llama-8B. https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Llama-8B.

[32] DeepSeek. 2025. DeepSeek-V3.2 Release. https://api-docs.deepseek.com/news/news251201.

[33] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 4171–4186.

[34] Danny Dolev and Andrew Yao. 2003. On the security of public key protocols. *IEEE Transactions on information theory* 29, 2 (2003), 198–208.

[35] Patrice Godefroid. 2020. Fuzzing: Hack, art, and science. *Commun. ACM* 63, 2 (2020), 70–76.

[36] Ali Güngör. 2026. UERANSIM: Open Source 5G UE and RAN (gNodeB) Simulator. https://github.com/aligungr/UERANSIM.

[37] Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A Smith. 2020. Don't stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th annual meeting of the association for computational linguistics*. 8342–8360.

[38] Yiwen Hu, Min-Yue Chen, Guan-Hua Tu, Chiyu Li, Sihan Wang, Jingwen Shi, Tian Xie, Li Xiao, Chunyi Peng, Zhaowei Tan, and Songwu Lu. 2022. Uncovering insecure designs of cellular emergency services (911). 703–715. doi:10.1145/3495243.3560534

[39] Yiwen Hu, Min-Yue Chen, Guan-Hua Tu, Chi-Yu Li, Sihan Wang, Jingwen Shi, Tian Xie, Li Xiao, Chunyi Peng, Zhaowei Tan, et al. 2022. Uncovering insecure designs of cellular emergency services (911). In *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking*. 703–715.

[40] Yiwen Hu, Min-Yue Chen, Guan-Hua Tu, Chi-Yu Li, Sihan Wang, Jingwen Shi, Tian Xie, Li Xiao, Chunyi Peng, Zhaowei Tan, and Songwu Lu. 2023. Unveiling the Insecurity of Operational Cellular Emergency Services (911): Vulnerabilities, Attacks, and Countermeasures. *GetMobile: Mobile Comp. and Comm.* 27, 1 (May 2023), 39–43. doi:10.1145/3599184.3599195

[41] Yiwen Hu, Min-Yue Chen, Haitian Yan, Chuan-Yi Cheng, Guan-Hua Tu, Chi-Yu Li, Tian Xie, Chunyi Peng, Li Xiao, and Jiliang Tang. 2024. Uncovering Problematic Designs Hindering Ubiquitous Cellular Emergency Services Access. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 1455–1469.

[42] Syed Hussain, Omar Chowdhury, Shagufta Mehnaz, and Elisa Bertino. 2018. LTEInspector: A systematic approach for adversarial testing of 4G LTE. In *Network and Distributed Systems Security (NDSS) Symposium 2018*.

[43] Syed Rafiul Hussain, Mitziu Echeverria, Omar Chowdhury, Ninghui Li, and Elisa Bertino. 2019. Privacy attacks to the 4G and 5G cellular paging protocols using side channel information. *Network and distributed systems security (NDSS) symposium2019* (2019).

[44] Syed Rafiul Hussain, Mitziu Echeverria, Imtiaz Karim, Omar Chowdhury, and Elisa Bertino. 2019. 5GReasoner: A property-directed security and privacy analysis framework for 5G cellular network protocol. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 669–684.

[45] Internet Systems Consortium. [n. d.]. BIND 9. https://www.isc.org/bind/.

[46] Georgios Kambourakis, Constantinos Kolias, Stefanos Gritzalis, and Jong Hyuk Park. 2011. DoS attacks exploiting signaling in UMTS and IMS. *Computer Communications* 34, 3 (2011), 226–235.

[47] Ronald Kemker, Marc McClure, Angelina Abitino, Tyler Hayes, and Christopher Kanan. 2018. Measuring catastrophic forgetting in neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 32.

[48] Hongil Kim, Jiho Lee, Eunkyu Lee, and Yongdae Kim. 2019. Touching the untouchables: Dynamic security analysis of the LTE control plane. In *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1153–1168.

[49] Leonard J LaPadula and D Elliot Bell. 1996. *Secure computer systems: A mathematical model.* Technical Report. Citeseer.

[50] Patrick PC Lee, Tian Bu, and Thomas Woo. 2009. On the detection of signaling DoS attacks on 3G/WiMax wireless networks. *Computer Networks* 53, 15 (2009), 2601–2616.

[51] Wai Kay Leong, Aditya Kulkarni, Yin Xu, and Ben Leong. 2014. Unveiling the hidden dangers of public ip addresses in 4g/lte cellular data networks. In *Proceedings of the 15th Workshop on Mobile Computing Systems and Applications*. 1–6.

[52] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[53] Yu-Han Lu, Chi-Yu Li, Yao-Yu Li, Sandy Hsin-Yu Hsiao, Tian Xie, Guan-Hua Tu, and Wei-Xun Chen. 2020. Ghost calls from operational 4G call systems: IMS vulnerability, call DoS attack, and countermeasure. In *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking*. 1–14.

[54] Benoit Michau and Christophe Devine. 2016. How to not break LTE crypto. In *ANSSI Symposium sur la sécurité des technologies de l'information et des communications (SSTIC)*.

[55] Netovate Ltd. 2025. 3GPP Change Request (CR) Search Tool. https://netovate.com/cr-search/.

[56] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.

[57] Palo Alto Networks. 2026. *What Are DNS Attacks?* Palo Alto Networks Cyberpedia. https://www.paloaltonetworks.com/cyberpedia/what-is-a-dns-attack

[58] Shinjo Park, Altaf Shaik, Ravishankar Borgaonkar, and Jean-Pierre Seifert. 2016. White rabbit in mobile: Effect of unsecured clock source in smartphones. In *Proceedings of the 6th Workshop on Security and Privacy in Smartphones and Mobile Devices*. 13–21.

[59] Qwen. 2025. Qwen3-4B-Instruct-2507. https://huggingface.co/Qwen/Qwen3-4B-Instruct-2507.

[60] Qwen. 2026. *Qwen-Plus Model Documentation.* https://modelstudio.console.alibabacloud.com/ap-southeast-1/?tab=doc#/doc/?type=model&url=2840914_2&modelId=qwen-plus

[61] Mirza Masfiqur Rahman, Imtiaz Karim, and Elisa Bertino. 2024. {CellularLint}: A systematic approach to identify inconsistent behavior in cellular network specifications. In *33rd USENIX Security Symposium (USENIX Security 24)*. 5215–5232.

[62] J. Rosenberg, H. Schulzrinne, G. Camarillo, A. Johnston, J. Peterson, R. Sparks, M. Handley, and E. Schooler. 2002. SIP: Session Initiation Protocol. RFC 3261. doi:10.17487/RFC3261

[63] Roshan Sedar, Charalampos Kalalas, Francisco Vázquez-Gallego, Luis Alonso, and Jesus Alonso-Zarate. 2023. A comprehensive survey of V2X cybersecurity mechanisms and future research paths. *IEEE Open Journal of the Communications Society* 4 (2023), 325–391.

[64] Altaf Shaik, Ravishankar Borgaonkar, N Asokan, Valtteri Niemi, and Jean-Pierre Seifert. 2015. Practical attacks against privacy and availability in 4G/LTE mobile communication systems. *arXiv preprint arXiv:1510.07563* (2015).

[65] Software Radio Systems. [n. d.]. srsRAN: Open Source LTE and 5G RAN Implementation. https://www.srsran.com/. Accessed: 2025-08-31.

[66] Sixu Tan, Zeyu Li, Zhutian Liu, Harsh Patel, and Zhaowei Tan. 2025. Automated Model-Based Fuzzing for 5G O-RAN. In *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking*. 201–215.

[67] Fabian Van Den Broek, Roel Verdult, and Joeri De Ruiter. 2015. Defeating IMSI catchers. In *Proceedings of the 22Nd ACM SIGSAC Conference on Computer and Communications Security*. 340–351.

[68] Sihan Wang, Guan-Hua Tu, Xinyu Lei, Tian Xie, Chi-Yu Li, Po-Yi Chou, Fucheng Hsieh, Yiwen Hu, Li Xiao, and Chunyi Peng. 2021. Insecurity of operational cellular IoT service: New vulnerabilities, attacks, and countermeasures. In *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*. 437–450.

[69] Chuan Yu and Shuhui Chen. 2019. On Effects of Mobility Management Signalling Based DoS Attacks Against LTE Terminals. In *2019 IEEE 38th International Performance Computing and Communications Conference (IPCCC)*. 1–8. doi:10.1109/IPCCC47392.2019.8958725

A New Vulnerabilities and Proof-of-concept Attacks

A.1 V3: Vulnerable DNS-Based PLMN Discovery over Non-3GPP Access

3GPP TS 23.501 defines the support for non-3GPP access, including the non-roaming architecture, Local Breakout (LBO) architecture, and Home-Routed (HR) roaming architecture. In these architectures, a UE can attach via an untrusted non-3GPP access network (e.g., untrusted satellite, Wi-Fi) in a visited network. To integrate such access, the UE's traffic is routed through the Non-3GPP Interworking Function (N3IWF) associated with the appropriate Public Land Mobile Networks (PLMN), enforcing access control and mobility anchoring.

To bootstrap 5G connectivity over untrusted non-3GPP access, the UE must discover an appropriate N3IWF. TS 23.501 specifies a DNS-based PLMN Discovery for non-3GPP access, where the UE performs a DNS NAPTR query to obtain service instance identifiers in the form of '<MNC>.<MCC>.pub.3gppnetwork.org'; the UE then uses DNS to resolve the IP addresses of N3IWF. This design is attractive from a deployment perspective because it reuses widely available DNS infrastructure without requiring pre-provisioned, static endpoint lists hardcoded on the UE. However, it also introduces a DNS attack surface. Prior measurements report that 88% of organizations have experienced DNS attacks [57]. Importantly, the specification does not mandate authenticated DNS transport or DNSSEC validation. Consequently, the DNS-based PLMN discovery responses can be spoofed or manipulated and an adversary can steer the victim toward the attacker-controlled or unreachable endpoints. While subsequent N3IWF procedures include cryptographic protection and authentication, this discovery-stage misdirection can create meaningful denial-of-service by preventing the UE from connecting to a legitimate N3IWF.

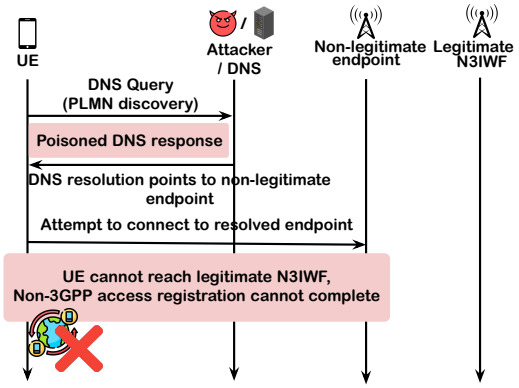


Figure 9: Vulnerable DNS-Based PLMN Discovery over Non-3GPP Access

A.1.1 Proof-of-concept Attack. We validate this vulnerability from two perspectives: (1) whether a DNS security mechanism (i.e., DNSSEC) is deployed on the operational domain name to protect the DNS-based PLMN Discovery mechanism, and (2) whether a DNS manipulation attack can practically prevent a UE from connecting to the legitimate N3IWF, as illustrated in Figure 9.

```
(.venv) test@test-TRX50-WS:~$ dig 3gppnetwork.org DNSKEY +dnssec

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> 3gppnetwork.org DNSKEY +dnssec
;; global options: +cmd
;; Got answer:                                          No DNSKEY
;; ->>HEADER<<- opcode: QUERY, status: SERVFAIL, id: 58674
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1
```

Figure 10: '3gppnetwork.org' does not enable DNSSEC

For (1), we first test whether the parent domain '3gppnetwork.org', managed by GSMA, is protected by DNSSEC, because DNSSEC validation for any descendant name (e.g., DNS-based PLMN Discovery) relies on a valid chain of trust.

Our measurement via the tool `dig` [45], as shown in Figure 10, indicates that the DNSSEC for '3gppnetwork.org' is absent. Therefore, DNS-based PLMN discovery remains vulnerable to classic DNS manipulation threats on untrusted links, even when subsequent procedures are cryptographically protected.

For (2), to evaluate whether the DNS manipulation can work, we test whether a poisoned discovery response can prevent the UE from establishing a secure tunnel to the legitimate N3IWF. Since conducting Internet-scale poisoning experiments is unethical, we perform experiments in a controlled environment. Specifically, we use two COTS UEs, Samsung S24 and Moto G Stylus 5G on T-Mobile network, and connect them to a controlled Wi-Fi router configured to poison DNS queries relevant to '3gppnetwork.org'. After enabling Wi-Fi calling services, we observe that both UEs try to resolve the IP address of N3IWF (i.e., ePDG for Wi-Fi calling services). Under the poisoned resolution, the UEs attempt to connect to the misdirected endpoint, thereby preventing the UE from reaching the legitimate N3IWF and completing non-3GPP access registration. This shows that, even though subsequent N3IWF procedures provide cryptographic protection and mutual authentication, compromising the discovery stage can still induce a denial-of-service attack.

A.1.2 Root Cause and Lesson Learned. The root cause is a trust gap in the DNS-based PLMN discovery, which is performed over untrusted access links before the UE establishes a secure channel to the operator's infrastructure. It is a standard design flaw that 3GPP specifications only protect the post-discovery connection and do not enforce security mechanisms during the discovery stage, which allows adversaries to exploit unauthenticated discovery to misdirect the victim. To address this vulnerability, future 3GPP specifications must enforce the security mechanisms on DNS-based PLMN discovery in two ways. First, discovery infrastructure must be treated as security-critical. Operational domains used for standardized discovery must provide an authenticated naming and resolution path. Second, standards need to mandate a robust fallback mechanism that avoids futile attempts.

A.2 V4: Privacy Exposure via Over-Disclosed VoIMS Signaling Headers

With the introduction of the all-IP network architecture in 4G LTE, call services previously supported by conventional circuit-switched (CS) technology in legacy 2G/3G networks have transitioned to being provided by the IMS. To facilitate IMS-based call services (also known as VoIMS - Voice over IMS), the Session Initiation Protocol (SIP), widely used in voice over IP services, was adopted for call signaling, replacing the legacy CS Call Control (CC) protocol. SIP

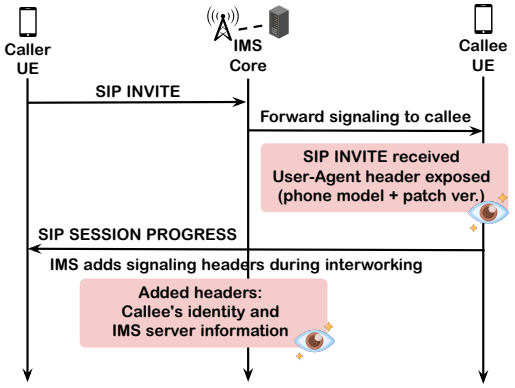


Figure 11: Privacy Exposure via Over-Disclosed VoIMS Signaling Headers

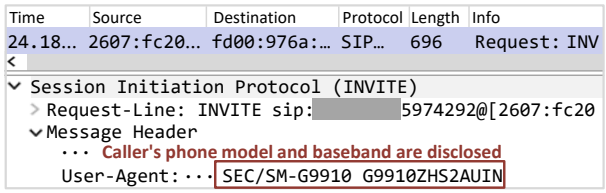
is a text-based, string-oriented protocol, in contrast to the binary, structure-oriented CS CC protocol.

Adopting a text-based call signaling protocol is a double-edged sword. While it offers greater flexibility and extensibility to support various emerging IP-based multimedia services compared to conventional protocols, it also introduces new issues due to potential missteps in design. Specifically, all 2G/3G CC signaling messages are designed to carry specific 3GPP-stipulated content only. In contrast, the text-based SIP protocol allows carriers and mobile devices to transmit a much broader range of content. Specifically, TS 24.229 stipulates 113 SIP headers and 15 SDP headers to support IMS-based call services, far more than its predecessor. These IMS service signaling headers can carry personal, device, and infrastructure details.

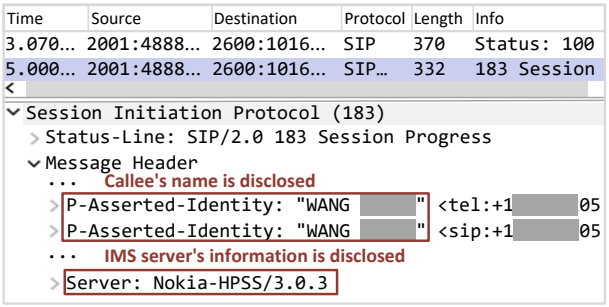
Unfortunately, as shown in Figure 11, there are no privacy preservation mechanisms enforced in 3GPP specifications for IMS signaling headers. While the signaling messages for normal IMS services are encrypted by IPSec, operators and UEs would carry such privacy-exposing headers to both IMS service users who can be the adversaries who collect the privacy information that may enable subsequent attacks.

A.2.1 Experimental Validation. We validate this potential privacy leakage by examining privacy leaks during call interworking. We control 2 phones, a Samsung S21 and a Google Pixel 5, on three US carriers (labeled as OP-I, OP-II, and OP-III) and collected signaling messages exchanged between two call ends. We make 10 calls for each combination of phone model and carrier. Our experimental results show that on the operational IMS services, privacy-leaking headers are transmitted and received by the UEs. Two privacy issues are explicitly identified in the messages SIP INVITE and SIP SESSION PROGRESS. In the former, as shown in Figure 12a, it is observed that the caller's User-Agent header is transmitted to the callee through the OP-II network, leaking the phone model and security patch version to the callee. For the latter as shown in Figure 12b, when the caller is on OP-III, and the callee is on OP-I, P-Asserted-Identity and Server headers are added by the networks in the SIP SESSION PROGRESS response, revealing the callee's identity and IMS server information.

A.2.2 Root Cause and Lesson Learned. Adopting a text-based call signaling protocol allows carriers and mobile devices to transmit



(a) User-Agent header leaks phone model and security patch version



(b) P-Asserted-Identity and Server headers leak callee's name and IMS server's information

Figure 12: The information about users and cellular infrastructure is leaked during call interworking

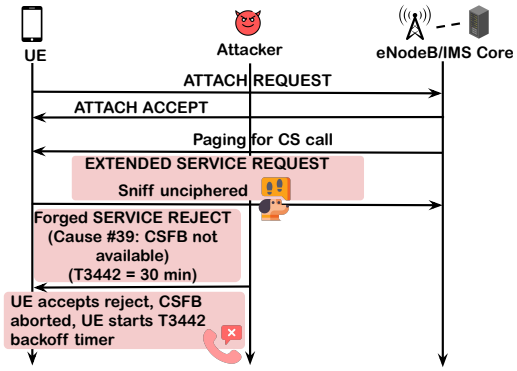


Figure 13: Exploitable CS-Fallback Initiation

a much broader range of content. While the 3GPP specification does not prohibit the transmission of these abundant headers, other network standard organizations like the IETF have raised concerns about privacy risks from unnecessary header disclosures. For example, RFC 3261 [62] warns about the vulnerability of disclosing user agent information. This underscores the urgent need for 3GPP to review and revise its IMS-related specifications with a focus on privacy preservation.

A.3 V5: Exploitable CS-Fallback Initiation

4G LTE networks support Circuit-Switched Fallback (CSFB) to provide legacy CS voice/SMS by redirecting a UE to a CS network (e.g., 2G/3G if available) when VoLTE is unavailable. Note that 5G non-standalone deployments that rely on 4G LTE infrastructure also support CSFB. In CSFB, after a UE in EMM-IDLE receives a paging request for a CS service, it initiates fallback by sending an EXTENDED SERVICE REQUEST to the Core Network. In the common case, the UE transitions from RRC_IDLE to RRC_CONNECTED and

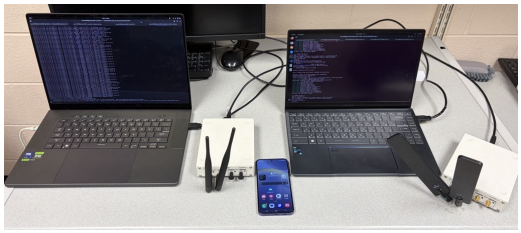


Figure 14: The setup for validating V5: Exploitable CS-Fallback Initiation

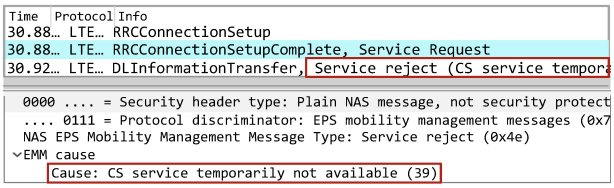


Figure 15: The victim UE accepts the rejection and ceases its attempts to establish the call

sends this request as an initial NAS message. In this case, the initial message is only integrity-protected without ciphering because NAS ciphering is only applied after the access-stratum connection and the relevant security context are re-activated for this specific exchange. Thus, adversaries can sniff the EXTENDED SERVICE REQUEST.

Moreover, TS 24.301 allows certain NAS reject messages, including SERVICE REJECT, to be sent and processed without any security protection. This combination exposes a practical CSFB DoS attack surface: adversaries can deny CSFB and rate-limit subsequent attempts by (1) sniffing unciphered EXTENDED SERVICE REQUEST and (2) injecting forged SERVICE REJECT with EMM cause #39 (CS domain temporarily not available) and the associated timer T3442 to throttle subsequent attempts.

A.3.1 Proof-of-concept Attack. Figure 13 illustrates a proof-of-concept attack. The attacker deploys a sniffer between the victim and the legitimate eNodeB. Once an unciphered EXTENDED SERVICE REQUEST is observed, the attacker immediately sends a forged SERVICE REJECT carrying EMM cause #39 and the T3442 via a rogue base station. To validate this attack, as shown in Figure 14, we implement an SDR-based testbed using srsRAN and open5GS [29], both of which support CS Fallback. A Samsung S24 and a Moto G Stylus are used as test UEs. Instead of building a full injection pipeline that monitors unciphered NAS messages and injects forged responses, which are credited to prior works [42], we simplify the evaluation by configuring Open5GS and srsRAN to emulate a CS call paging and then directly respond with a SERVICE REJECT with EMM cause #39 and T3442 for the maximum value 30 mins directly. As shown in Figure 15, the experimental traces indicate that the UEs accept the rejection and stop attempting to set up the call. Overall, this attack achieves a stealthy denial-of-service against CS services on LTE by preventing a CSFB procedure, and limiting further CSFB attempts via T3442. The throttling effect is particularly damaging because the UE self-enforces the backoff timer. Depending on UE implementation and timer configuration, the victim may experience prolonged inability to place CS calls. We leave a systematic characterization of the maximum outage duration to future work.

A.3.2 Root Cause and Lesson Learned. The root cause is TS 24.301 explicitly permits a SERVICE REJECT to be processed without ciphering and integrity protection. This design appears risky but serves as a fail-safe mechanism for cases where a secure connection cannot be established or has been lost. For example, if a UE sends an EXTENDED SERVICE REQUEST but the MME cannot authenticate the user (e.g., the unknown/invalid subscription), the network must be able to instruct the UE to stop trying without security keys. Similarly, for emergency services, the network must always be able to reject or redirect requests even if security procedures are failing to ensure the UE can seek alternative access. To mitigate this threat, commonly from rogue base stations, and still allow certain unprotected signaling messages for a fail-safe mechanism, in practice, it likely requires UE-side defenses against rogue base stations and stricter validation before setting the long throttling timers.

A.4 V6: Access Trap via Access Barring Factor

In 5G NR, System Information Block 1 (SIB1) is a periodically broadcast RRC message that every UE must decode first before any AS (Access Stratum) security is established. SIB1 carries essential cell-access and cell-selection parameters, including PLMN, scheduling of other system information blocks, and uac-BarringInfo for UAC (User Access Control). uac-BarringInfo specifies barring rules for different access categories, distinguishing everyday services (e.g., regular voice/data and mobile-originated signaling) from special-case/high-priority categories (e.g., paging/mobile-terminated access and emergency access). A similar design also existed in 4G networks, which used the ac-BarringInfo field in SIB2 messages.

Each barring rule includes parameters such as barringFactor, which controls the probability that a UE is allowed to initiate an access attempt for a given category. Intuitively, a smaller barringFactor imposes a stronger restriction, while an extremely small value can cause most or nearly all UEs to defer their mobile-originated requests. In normal operation, base stations typically configure moderate barring parameters, or disable barring entirely, to preserve broad service availability and avoid unnecessarily suppressing legitimate access.

This design becomes exploitable when a malicious base station advertises an extremely restrictive barringFactor in SIB1/SIB2 in 5G/4G networks. After decoding the broadcast information, the UE enforces the barring policy locally. By setting barringFactor to a highly restrictive value, the attacker can effectively drive victim UEs into a stealthy nonfunctional state, where the device appears camped on a cell but repeatedly refrains from initiating normal services, thereby creating a stealthy denial-of-service condition.

A.4.1 Proof-of-concept Attack. Figure 16 illustrates the proof-of-concept attack. The adversary only needs to operate a rogue base station and exploit broadcast system information; no cryptography is broken. Specifically, the adversary needs to configure the rogue base station to broadcast SIB1 in 5G/SIB2 in 4G with barringFactor set to a highly restrictive value (e.g., 0), which sets a low probability that a UE is allowed to initiate an access attempt. Therefore, any nearby UEs that receive and decode this SIB1/SIB2 are not able to initiate an access attempt.

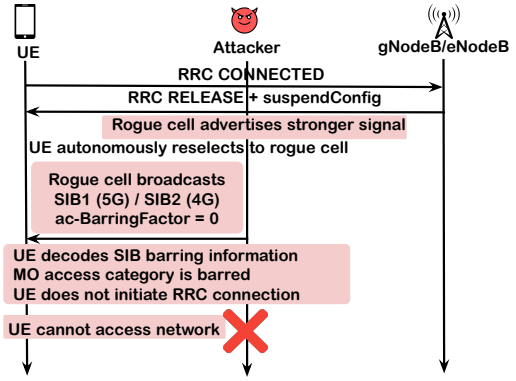


Figure 16: Access Category Trap via SIB1 Access Barring

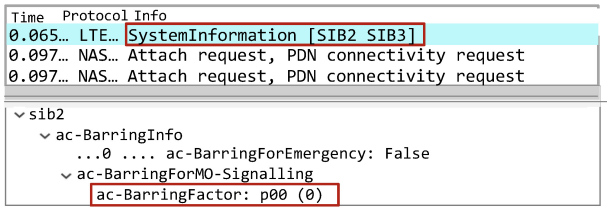


Figure 17: The SIB2 messages broadcast by attackers prevent UEs from attaching to base stations

To validate this vulnerability ethically, we implement an SDR-based testing platform using srsRAN and srsEPC. All antennas are placed in a shield box to ensure that no unintended over-the-air transmissions occur. We use a Samsung S24 and a Moto G Stylus 5G as test phones. Next, we simplify the experiment and configure the controlled serving cell to broadcast SIB2 with an ac-BarringFactor setting to 0. As shown in Figure 17, we observe (i) the UE does not initiate RRC connection establishment after receiving a SIB2 with BarringFactor setting to 0, and (ii) the data service on UEs is not usable without interfaces. Our experimental results show that both phones self-block service without explicit alerting beyond generic 'no service' icon, consistent with the intended UAC enforcement behavior but enabling a stealthy denial-of-service.

A.4.2 Root Causes and Lesson Learned. UAC is designed for congestion control: the UE locally enforces access restrictions to reduce network load and stabilize access attempts during overload. However, the policy is delivered via SIB1/SIB2, which is broadcast and processed prior to authentication and therefore lacks integrity protection. This creates a pre-authentication trust gap: a rogue cell can advertise restrictive UAC policies and rely on the UE to self-enforce them, achieving denial-of-service without needing to defeat cryptographic protections. To mitigate this vulnerability, 3GPP standards shall develop UE implementations that can reduce exposure by adding safety checks and recovery logic (e.g., bounding the duration of extreme barring, triggering reselection/blacklisting when a newly camped cell persistently bars most categories, or cross-validating with neighboring cells).

A.5 V7: Rogue Base Station RRC RESUME Blackhole

In 5G NR, a UE selects the cell based on the broadcast radio information and measured radio conditions before any AS/NAS security

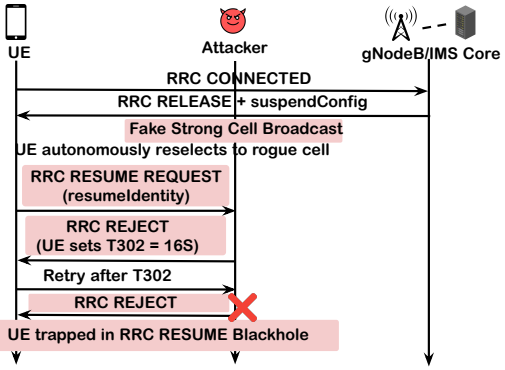


Figure 18: Rogue Base Station RRC RESUME Blackhole

context is activated. Concretely, parameters conveyed via the broadcast messages (e.g., SSB/PBCH, MIB, and SIBs) are not authenticated at the UE during this procedure. As a result, a rogue base station can attract a UE through (1) stronger radio conditions in terms of RSRP, RSRQ, and SINR, (2) advertising the legitimate PLMN info that the UE is willing to use, and (3) advertising that it is not barred for achieving a high priority in the standard cell selection and reselection mechanism defined in TS 36.304.

Combining the cell selection and reselection mechanism with the RRC RESUME procedure that a UE in RRC INACTIVE expects to resume quickly via RRC RESUME REQUEST, the UE can be induced to reselect into a gNB that does not recognize or even a rogue gNB that can intentionally ignore or reject this request. This resume thus fails. Moreover, instead of directly switching to another cell, TS 36.304 [8] defines that a UE should retry after a wait time specified by the timer T302. The cell selection and reselection mechanism does not consider the resume failure. Thus, the victim UE will stick to the malicious base station until a legitimate gNB wins in the cell selection and reselection procedure. While 5G security prevents a rogue gNB from completing an authenticated service, repeated resume failures can still cause meaningful availability harm for the retry loops.

A.5.1 Proof-of-concept Attack. We refer to the following behavior as an *RRC RESUME Blackhole*: the victim UE is trapped within a rogue gNB and stuck in repeated RRC RESUME REQUEST→ RRC RESUME failure cycles. The attacker does not need to break cryptography and does not need to complete mutual authentication. Instead, the attacker exploits (i) unauthenticated cell selection and (ii) the coupling between reselection and the RRC RESUME mechanism.

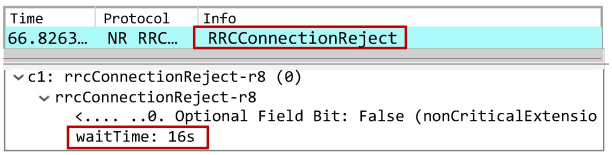


Figure 19: The victim UE remains unable to resume services and repeats the procedure every 16 seconds according to the timer.

For ethical reasons, we validate this attack in a fully controlled environment. First, inside an RF shield box, we deploy two SDR-based gNBs using srsRAN and open5GS: One serves as a legitimate

Table 7: Comparison of existing systems for 3GPP specification security analysis

Method	Primary Objective	Protocol Modeling	Security Scope	Remarks
Atomic [25]	Detecting hazard indicators via textual entailment	×	Risk-sentence detection only	Identifying hazard indicators via sentence-level textual entailment without modeling specification logic or performing systematic vulnerability analysis
ConTester [24]	Extracting security requirements	×	Developing conformance test cases	Only extracting security requirements for developing conformance test without uncovering design-level vulnerabilities
CellularLint [61]	Detecting semantic inconsistencies	×	Sentence-level contradiction detection	Detecting semantic inconsistencies (i.e., sentence-level contradictions) via sentence-pair inference instead of reasoning over complete specification logic
Hermes [14]	Automatic FSM extraction via traditional NLP approaches	✓	Formal security analysis	Parsing specifications via handcrafted grammars using constituency parsing and dependency parse trees, remaining rule-intensive and lacking flexibility to adapt to evolving specifications
ARCANE [66]	Automated vulnerability discovery via model-based fuzzing	✓	Implementation vulnerability discovery	Automatically constructs protocol models and performs model-based fuzzing
CellSecInspector	End-to-end SCA nodes and systematic vulnerability reasoning	✓	Specification-level design-flaw detection	First system to enable scalable specification extraction and systematic security analysis

gNB, and another serves as the rogue gNB. The rogue gNB is configured to have same settings as the legitimate one. Two test phones, a Samsung S21 and a Moto G Stylus 5G, are employed. Second, we increase the rogue gNB signal strength via increasing its tx_gain and rx_gain in srsRAN so that the rogue gNB can be the first choice in the cell selection and reselection mechanism. Third, we operate the test phones to trigger the RRC RESUME. Fourth, we configure the rogue gNB to respond with RRC REJECT for all RRC RESUME REQUEST with a RejectWaitTime 16, which will be the time used by the UE as T302. Last, we examine if the UE is stuck to the rogue gNB and trapped within RRC RESUME Blackhole. Figure 19 illustrates a trace captured on the Moto G Stylus 5G, which remains unable to resume service for 16 seconds after receiving RRC REJECT. The Samsung S21 exhibits a different implementation behavior: it does not strictly follow the 3GPP-specified wait time and stops sending RRCResumeRequest for only about 3 seconds even when RejectWaitTime = 16 is provided. This result shows that although the attack is effective on both devices, UE implementations may differ in how strictly they enforce the backoff timer.

A.5.2 Root Causes and Lessons Learned. The root cause is that the cell selection and reselection mechanism developed in 3GPP standards does not consider these failures. It operates on the RRC layer before the UE can establish a protected AS channel. Future specifications and deployments can consider additional mechanisms to reduce resume failures (e.g., explicit guidance for UEs to switch from cells that cannot support resumption).

B Related Work

Table 7 summarizes and contrasts representative 3GPP specification analysis systems in terms of their primary objectives, protocol modeling capabilities, security analysis scope, and key methodological characteristics.

C *CellSecInspector*

This appendix provides implementation-level details of the prompts and algorithms used by *CellSecInspector*. It covers *SpecAdaptation*, the *SCA Representation Extractor*, the *Function Chain Builder*, *SecOracle*, and *VulnTestGenerator*. The Function Chain Builder is formalized through Algorithm 2, Algorithm 3, Algorithm 4, and Algorithm 5, which respectively define Temporal, Semantic, Causal, and Reference Guided Connection.

C.1 SpecAdaptation

Algorithm 1 *SpecAdaptation*

Input: Versioned 3GPP corpus $\mathcal{D}$, component query q, prompt template $\mathcal{P}$, retrieval size k
Output: Grounded prompt τ for *CellSecInspector-Core*
/ Step 1: prepare a versioned specification index. */*
$\mathcal{D}' \leftarrow \textsc{Sanitize}(\mathcal{D})$
/ Remove formatting noise while preserving specification number, release/version, clause ID, and cross-references. */*
$\mathcal{I} \leftarrow \textsc{BuildIndex}(\mathcal{D}')$
/ Each indexed chunk keeps its source metadata for traceability. */*
/ Step 2: retrieve evidence for the current component. */*
$\mathcal{E} \leftarrow \textsc{RetrieveTopK}(q, \mathcal{I}, k)$
/ The query may come from a clause, message type, a timer, an SCA node pair, security property, or candidate violation. */*
/ Step 3: expand evidence through explicit cross-references. */*
for each retrieved chunk $e \in \mathcal{E}$ **do**
  $C_e \leftarrow \textsc{ExtractReferencedClauses}(e)$
  if $C_e \neq \emptyset$ **then**
    $\mathcal{E} \leftarrow \mathcal{E} \cup \textsc{RetrieveClauses}(C_e, \mathcal{I})$
    / Include referenced clauses such as "as specified in subclause X.Y.Z". */*
  end if
end for
/ Step 4: attach metadata for grounded reasoning. */*
$\mathcal{M} \leftarrow \textsc{GetMetadata}(\mathcal{E})$
/ Metadata includes specification number, release/version, clause ID, and source location. */*
/ Step 5: assemble the grounded prompt for the downstream module. */*
$\tau \leftarrow \textsc{AssemblePrompt}(\mathcal{P}, q, \mathcal{E}, \mathcal{M})$
/ The prompt contains task instructions, structured input, retrieved specification evidence, and required output schema. */*
return τ

SpecAdaptation prepares the specification knowledge used by all later modules. It first sanitizes the collected 3GPP specifications by removing formatting noise while preserving clause numbers, cross-references, and release/version identifiers. It then builds a versioned retrieval index so that *CellSecInspector-Core* can reason over the relevant specification text rather than relying only on its parametric memory. Given a component query, such as a clause, an SCA node, a candidate node pair, or flagged violation, *CellSecInspector-Core* retrieves relevant specification excerpts from the indexed corpus and attaches them to the prompt together with clause identifiers and release/version information.

For example, when extracting an SCA node from the TS 24.501 sentence "upon receipt of a message of a network-initiated 5GMM

common procedure, the UE shall stop timer T3540 and respond to the network-initiated 5GMM common procedure via the existing N1 NAS signalling connection as specified in subclause 5.4," *SpecAdaptation* retrieves the source clause containing this sentence and the referenced subclause 5.4. The retrieved evidence allows *CellSecInspector-Core* to ground the extraction in the analyzed TS 24.501 version, infer that the UE has an active N1 NAS signalling connection and timer T3540 is running, and extract the action that the UE stops T3540 and responds via the existing N1 NAS signalling connection. Algorithm 1 summarizes this workflow.

C.2 SCA Representation Extractor

The *SCA Representation Extractor* maps specification text into structured SCA nodes. Each node contains four fields: start state, condition, action, and end state. The prompt in Figure 20 instructs *CellSecInspector-Core* to extract these fields from the retrieved specification evidence. If the specification text does not explicitly state a field, *CellSecInspector-Core* is instructed to infer it only when the surrounding context supports the inference; otherwise, the field is marked as not explicitly defined. This design preserves both explicit specification semantics and traceable best-effort inference.

C.3 Function Chain Builder

After SCA nodes are extracted, the *Function Chain Builder* connects scattered nodes into function chains. Temporal Connection links two nodes when the end state of one node exactly matches the start state of another node, as formalized in Algorithm 2. Semantic Connection links two nodes when their end/start states are textually different but semantically equivalent, as described in Algorithm 3. Causal Connection links two nodes when one node's state, condition, or action enables another node, as shown in Algorithm 4. Reference Guided Connection uses explicit cross-references in 3GPP specifications to narrow the search space before applying the above checks, as detailed in Algorithm 5.

C.4 SecOracle

Given a function chain and an adversarial action, the prompt in Figure 21 asks *SecOracle* to determine whether dropping, modifying, injecting, or replaying a message or event can produce a realistic candidate violation.

Subject Assignment Verification. We introduce a prompt-based actor and message-direction check before evaluating the security consequence. *SecOracle* first identifies the attacked message and determines its sender and receiver from the corresponding SCA nodes and signaling direction. It then constrains receiver-side operations, such as processing, accepting, rejecting, validating, or handling a message, to the receiving entity rather than the sender. Specifically, for a UE-to-network message, the consequence is analyzed at the receiving AMF, MME, or other network entity; for a network-to-UE message, the consequence is analyzed at the receiving UE. If the message direction or involved entities cannot be determined from the specification evidence, *SecOracle* is instructed to report the uncertainty rather than infer an unsupported actor.

The prompt also checks whether a valid security context, integrity verification, freshness mechanism, and replay protection are explicitly active at the analyzed transition. This prevents *SecOracle* from assuming that a message is protected merely because such protection exists elsewhere in the protocol.

SCA Representation Extractor

You are a mobile network security test engineer specializing in 3GPP protocols, including 5G and 4G NAS, RRC, IMS/SIP signaling, and related mobile network procedures. Your task is to analyze the given sentence and extract an SCA node.

SCA node format:
Node ID: {node_id}{section_info}
Sentence: "{sentence}"
Start State: Representing the initial system state before the transition.
Condition: Denoting the triggering clause or prerequisite specified in the specification.
Action: Describing the operation mandated by the specification once the condition is satisfied.
End State: indicating the resulting system state after the action is executed.

Rules:
1. Each sentence corresponds to exactly ONE SCA node.
2. Fill in **all fields (Start State, Condition, Action, End State).**
 - If the sentence only serves as a structural heading, use **"Not specified"** for all fields.
 - If a field is unclear, use **"Not explicitly defined"** instead of leaving it blank.
3. Retain all 3GPP references (e.g., **"as specified in subclause X.Y.Z"**).
4. Do NOT rephrase the sentence; keep it as provided.
5. Ensure consistency, always follow the event format exactly.

Example:
1. Input Sentence: The non-access stratum (NAS) described in the present document forms the highest stratum of the control plane between UE and AMF (reference point "N1" see 3GPPTS23.501[8]) for both 3GPP and non-3GPP access.
Expected Output:
Event ID: 5 (Derived from Section 4.1)
Sentence: "The non-access stratum (NAS) described in the present document forms the highest stratum of the control plane between UE and AMF (reference point "N1" see 3GPPTS23.501[8]) for both 3GPP and non-3GPP access"
Start State: UE and AMF are not explicitly connected at the NAS level.
Condition: UE needs to communicate with AMF using NAS procedures.
Action: Define NAS as the highest control plane layer and establish its role in UE-AMF interaction.
End State: NAS is identified as the highest control plane stratum for both 3GPP and non-3GPP access.

2. If required by operator policy, the AMF shall include the NSSAI inclusion mode IE in the REGISTRATION ACCEPT message (see table 4.6.2.3.1 of subclause 4.6.2.3). Upon receipt of the REGISTRATION ACCEPT message.
Expected Output:
Event ID: 5675 (Derived from Section 5.5.1.3.4)
Sentence: "If required by operator policy, the AMF shall include the NSSAI inclusion mode IE in the REGISTRATION ACCEPT message (see table 4.6.2.3.1 of subclause 4.6.2.3). Upon receipt of the REGISTRATION ACCEPT message"
Start State: The UE has sent a REGISTRATION REQUEST message to the AMF, and the AMF is processing the registration request.
Condition: Operator policy requires the inclusion of the NSSAI inclusion mode IE in the REGISTRATION ACCEPT message.
Action: The AMF includes the NSSAI inclusion mode IE in the REGISTRATION ACCEPT message as specified in table 4.6.2.3.1 of subclause 4.6.2.3.
End State: The REGISTRATION ACCEPT message, containing the NSSAI inclusion mode IE, is sent to the UE.

Figure 20: ICL instruction for *SCA Representation Extractor*

The output is a structured candidate-violation report containing the candidate-violation decision, potentially violated security requirements, explanation, issue classification, validation test idea, and validation source node.

Algorithm 2 Temporal Connection

Input: Set of SCA nodes $N = \{n_1, \ldots, n_m\}$
Output: Temporal-Connection relation T
$T \leftarrow \emptyset$
/ Temporal connection is a strict state-matching rule. */*
/ It links n_i to n_j only when the end state of n_i is the same as the start state of n_j. */*
for each ordered pair $(n_i, n_j) \in N \times N$ **do**
  $e_i \leftarrow \text{end}(n_i)$
  $s_j \leftarrow \text{start}(n_j)$
  / Skip underspecified states because they cannot support reliable temporal matching. */*
  if $e_i = s_j$ **and** $e_i \notin$ {"not specified", "not explicitly defined"} **then**
    $T \leftarrow T \cup \{(n_i, n_j)\}$
    / n_j is a valid temporal successor of n_i. */*
  end if
end for
return T

Algorithm 3 Semantic Connection

Input: Set of SCA nodes $N = \{n_1, \ldots, n_m\}$, model *CellSecInspector-Core*
Output: Semantic-Connection relation S
$S \leftarrow \emptyset$
/ Semantic connection handles cases where two states are equivalent but written differently. */*
for each ordered pair $(n_i, n_j) \in N \times N$ **do**
  $e_i \leftarrow \text{end}(n_i)$
  $s_j \leftarrow \text{start}(n_j)$
  if e_i and s_j are valid **then**
    / Step 1: Encode the two state descriptions. */*
    $\mathbf{t}_i \leftarrow \textsc{Tokenize}(e_i)$
    $\mathbf{t}_j \leftarrow \textsc{Tokenize}(s_j)$
    $\mathbf{H}_i \leftarrow$ *CellSecInspector-Core*.$\textsc{Encode}(\mathbf{t}_i)$
    $\mathbf{H}_j \leftarrow$ *CellSecInspector-Core*.$\textsc{Encode}(\mathbf{t}_j)$
    $\mathbf{z}_i \leftarrow \textsc{MeanPool}(\textsc{LastLayer}(\mathbf{H}_i))$
    $\mathbf{z}_j \leftarrow \textsc{MeanPool}(\textsc{LastLayer}(\mathbf{H}_j))$
    / Step 2: Estimate textual semantic similarity. */*
    $sim \leftarrow \textsc{Cosine}(\mathbf{z}_i, \mathbf{z}_j)$
    / Step 3: Ask CellSecInspector-Core to verify whether the two states represent the same protocol status. */*
    $\tau \leftarrow \textsc{AssemblePrompt}(\mathcal{P}_{sem}, e_i, s_j, sim)$
    $r \in \{\text{Yes}, \text{No}\} \leftarrow$ *CellSecInspector-Core*(τ)
    if $r =$ Yes **then**
      $S \leftarrow S \cup \{(n_i, n_j)\}$
      / n_j is linked as a semantic successor of n_i. */*
    end if
  end if
end for
return S

Algorithm 4 Causal Connection

Input: Set of SCA nodes $N = \{n_1, \ldots, n_m\}$, model *CellSecInspector-Core*
Output: Causal-Connection relation C
$C \leftarrow \emptyset$
/ Causal connection captures enabling relations that cannot be reduced to exact state matching. */*
/ For example, the action of one node may establish a context, timer, or connection required by another node. */*
for each ordered pair $(n_i, n_j) \in N \times N$ **do**
  / Step 1: Collect the full SCA fields of both nodes. */*
  $x_i \leftarrow \{\text{start}(n_i), \text{condition}(n_i), \text{action}(n_i), \text{end}(n_i)\}$
  $x_j \leftarrow \{\text{start}(n_j), \text{condition}(n_j), \text{action}(n_j), \text{end}(n_j)\}$
  / Step 2: Build a reasoning prompt with the causal definition and decision rules. */*
  $\tau \leftarrow \textsc{AssemblePrompt}(\mathcal{P}_{causal}, x_i, x_j)$
  / Step 3: CellSecInspector-Core decides whether n_i enables, triggers, or makes n_j reachable. */*
  $r \in \{\text{Yes}, \text{No}\} \leftarrow$ *CellSecInspector-Core*(τ)
  if $r =$ Yes **then**
    $C \leftarrow C \cup \{(n_i, n_j)\}$
    / n_j is linked as a causal successor of n_i. */*
  end if
end for
return C

Algorithm 5 Reference Guided Connection

Input: Set of SCA nodes $N = \{n_1, \ldots, n_m\}$, clause-to-node index $\mathcal{M}$
Output: Reference Guided relation R
$R \leftarrow \emptyset$
/ Reference Guided Connection reduces the search space using explicit cross-references in specifications. */*
/ It avoids comparing all node pairs when the text already points to relevant clauses. */*
for each node $n_i \in N$ **do**
  / Step 1: Detect references such as "as specified in subclause X.Y.Z". */*
  $C \leftarrow \textsc{ExtractReferencedClauses}(n_i)$
  if $C = \emptyset$ **then**
    continue
  end if
  / Step 2: Retrieve only nodes derived from the referenced clauses. */*
  $\mathcal{N}_c \leftarrow \textsc{RetrieveCandidateNodes}(C, \mathcal{M})$
  / Step 3: Validate whether each referenced candidate forms a valid connection. */*
  for each candidate $n_j \in \mathcal{N}_c$ **do**
    if $\textsc{TemporalCheck}(n_i, n_j)$ **or** $\textsc{SemanticCheck}(n_i, n_j)$ **or** $\textsc{CausalCheck}(n_i, n_j)$ **then**
      $R \leftarrow R \cup \{(n_i, n_j)\}$
      / A reference-guided edge is added only after passing at least one connection check. */*
    end if
  end for
end for
return R

C.5 VulnTestGenerator

After *SecOracle* flags a candidate violation, *VulnTestGenerator* converts the candidate-violation report into a validation-oriented test case. The prompt in Figure 22 instructs *CellSecInspector-Core* to specify the required network state, UE state, security-context status, attacker capability, operation sequence, and expected observation. The generated test case does not by itself execute the experiment; instead, it provides a reproducible procedure that guides validation on a controlled testbed or operational network when ethical and practical constraints permit.

D Implementation and Datasets

We use a workstation running Ubuntu 24.04 LTS with an NVIDIA GeForce RTX 4090 GPU (CUDA 12.6; NVIDIA driver 560.35.03) as the primary client environment to access remote compute resources and run supporting experiments. 4 LLMs, DeepSeek-V3.2 [32], Qwen-Plus [60], DeepSeek-R1-8B [31], and Qwen3-4B [59] are used as the base models in the evaluation. We access the first two models through their public APIs. The later two are self-hosted on Jetstream2 [4] using two H100s. DeepSeek-V3.2-Release is primarily used as the base model to build *CellSecInspector-Core*. The implementations for vulnerability validation differ across vulnerabilities; we describe the corresponding setups and procedures in the relevant vulnerability sections.

Specification Dataset. To ensure comparability with prior works, we construct a dataset from the same 3GPP specifications that govern control-plane signaling for NAS and RRC, which are critical for the security and mobility of operational 5G and 4G networks. In addition, to showcase that *CellSecInspector* can be applied to other 3GPP standards, specifications for 5G system architecture and SIP/SDP protocols are also included. Thus, the dataset includes: ***TS 38.331*** version 17.0.0 Release 17 [11], ***TS 24.501*** version 16.8.0 Release 16 [10], ***TS 24.301*** version 16.8.0 Release 16 [9], ***TS 23.501*** version 20.0.0 Release 20 [12], and ***TS 24.229*** version 19.5.0 Release 19 [13].

SecOracle

You are a mobile network security test engineer specializing in 3GPP protocols, including 5G and 4G NAS, RRC, IMS/SIP signaling, and related mobile network procedures. Evaluate whether an {attack_type} attack between Event {from_id} ({from_event}) and Event {to_id} ({to_event}) poses a realistic, meaningful candidate violation.
Attack scenario: A(n) {attack_type} attack may occur at some point between these transitions.
Actor and message-direction rules:
- Before describing an attack effect, identify the attacked message, its sender, and its receiver.
- The subject of verbs such as "process", "accept", "reject", "validate", or "handle" must be the receiver of the attacked message, not the sender.
- For UE-to-network messages, analyze effects on the receiving AMF/MME/network entity; for network-to-UE messages, analyze effects on the receiving UE.
- A replay attack may target any transmitted message and does not need to trigger the sender's own state transition.
- If the sender, receiver, or protection status is unclear, state the uncertainty instead of guessing.
Security-context and protection-status rules:
- Do not assume that a NAS message is integrity-protected, ciphered, or replay-protected merely because it is a NAS message.
- Before dismissing a replay/modify/inject attack, determine whether a valid security context, freshness mechanism, and integrity verification are explicitly active in the given transition.
- Pay special attention to transitions involving registration, service request, idle/connected switching, RRC inactive/suspend/resume, security context establishment, or security mode procedures, where protection may be absent, partial, stale, or not yet validated.
- If the transition does not explicitly show that integrity and freshness checks are active before message processing, do not use them as the sole reason to mark the attack as non-violation.
- Distinguish between protection that exists in the protocol in general and protection that is actually available at this transition point.
Relevant security properties and requirements: {sec_list}
Evaluation criteria:
- Respond "candidate_violation": "Yes" **only if** the attack introduces a realistic violation with observable consequences.
- Violations may include: Authentication bypass, Integrity protection failure, State inconsistency between UE and network, Session hijacking or denial of service, Message replay that causes real state divergence
- Do not respond "Yes" merely because a message is optional or dropped, **but** if this optional handling creates inconsistencies, opens the door to spoofed messages, or breaks expected session behavior, then you **must** consider it a violation.
Return your answer strictly in the following JSON format:
BEGIN_JSON
{{
"candidate_violation": "Yes" or "No",
"violated_requirements": [list of integers],
"explanation": "short explanation",
"issue_classification": "Protocol Design Issue" or "Implementation Issue" or "Both" or "N/A",
"test_case": "short string or 'N/A'",
"candidate_violation_source": "event{from_id}" or "event{to_id}" or "event{from_id} and event{to_id}" or "unclear"
}}
END_JSON
"""

Figure 21: *SecOracle*

VulnTestGenerator

You are a mobile network security test engineer specializing in 3GPP protocols, including 5G and 4G NAS, RRC, IMS/SIP signaling, and related mobile network procedures.

Your task is to design a concrete validation procedure for the given attack scenario and output it as a Markdown table with the following columns:

Step	Procedure	U–M	Message	Parameter	Verdict

Definitions:
* Step:
  Integer starting from 1 and strictly increasing.
* Procedure:
  A short sentence describing what happens in this step.
* U–M:
  "→" if the logical protocol direction is from the UE to the mobile network.
  "←" if the logical protocol direction is from the mobile network to the UE.
  "-" if no UE-to-network or network-to-UE message is exchanged.
* Message:
  The main protocol message name(s) or event involved in this step, such as a NAS, RRC, SIP, or other relevant signaling message, or "-" if none.
* Parameter:
  Important parameters, information elements (IEs), flags, timers, or other protocol fields involved in this step, or "-" if none.
* Verdict:
  "-" for intermediate steps.
  Only the final step should contain a verdict.
  Use "Fail" if the tested security property is violated or the attack successfully produces the target violation.
  Use "Pass" if the expected security property remains satisfied and the attack does not produce the target violation.

Attack type: {attack}

node_i (from event):

* Start State: {node_i.Start State}
* Condition: {node_i.Condition}
* Action: {node_i.Action}
* End State: {node_i.End State}

node_j (to event):

* Start State: {node_j.Start State}
* Condition: {node_j.Condition}
* Action: {node_j.Action}
* End State: {node_j.End State}

SecOracle output JSON describing the candidate violation and test idea:

{json_blob}

Return only the completed Markdown table. Do not include additional explanation before or after the table.

Figure 22: *VulnTestGenerator*

E Security Properties

E.1 Authentication

Ensures that only legitimate users, devices, and network entities can access services.

- **UE Authentication:** Ensures that only authorized users can access the network.
- **Network Entity Authentication:** Verifies the legitimacy of network nodes (eNodeB, gNodeB, AMF, SMF, etc.).
- **Mutual Authentication:** Guarantees two-way authentication between the UE and the network to prevent man-in-the-middle attacks.
- **SBA Authentication Binding:** Enforces token-based authentication between Service-Based Architecture (SBA) functions.
- **Secure PLMN Selection:** Protects against forced redirection to rogue or unauthorized Public Land Mobile Networks (PLMNs).
- **Roaming & Inter-PLMN Security:** Ensures consistent authentication policies across operators during roaming.

E.2 Authorization

Controls permissions and restricts operations to authorized entities only.

- **API & Service Access Control:** Enforces strict authorization for APIs and service-based interfaces.
- **Policy & QoS Rule Protection:** Prevents unauthorized modification of policies, Quality of Service (QoS) rules, or charging rules.
- **Edge Application Security:** Ensures strong isolation between Multi-access Edge Computing (MEC) applications.
- **SBA API Security:** Requires strong authorization and input validation for Service-Based Architecture (SBA) REST APIs.
- **Slice Isolation:** Ensures strict separation between 5G network slices (eMBB, URLLC, mMTC, etc.).
- **Multi-Tenancy Isolation:** Guarantees tenant isolation in SBA and MEC environments.
- **Network Function Isolation:** Enforces logical separation between core network functions (AMF, SMF, UPF, etc.).

E.3 Service Integrity

Guarantees that services, signaling, and data remain correct, unaltered, and trustworthy.

- **Message Integrity:** Ensures signaling and data messages are not altered during transmission.
- **Security Context Validation:** Ensures NAS/RRC security contexts are properly established before any sensitive exchanges.
- **Secure Paging Mechanisms:** Validates paging messages and prevents spoofed triggers.
- **Secure Emergency Services:** Ensures integrity-protected emergency registration and deregistration procedures.

E.4 Service Confidentiality

Protects sensitive signaling and user data from unauthorized access or disclosure.

- **Data Encryption:** Protects user data and signaling from interception.
- **Service Confidentiality:** Ensures sensitive information remains protected at all times.
- **Integrity & Confidentiality Binding:** Ensures both encryption and integrity protections are activated together.
- **Secure Algorithm Negotiation:** Prevents downgrades to weak ciphering or integrity algorithms.
- **Session Key Update:** Secures key re-derivation and renewal during mobility or context switches.
- **Key Lifecycle Management:** Ensures secure generation, distribution, update, and revocation of cryptographic keys.

E.5 Privacy Protection

Prevents leakage of sensitive identifiers, location information, and other personal data.

- **Identifier Protection:** Ensures SUPI, SUCI, GUTI, and other permanent identifiers are never exposed in plaintext.
- **Location Privacy:** Protects against user tracking through paging, tracking area (TA) updates, or broadcast messages.
- **Secure Paging Privacy:** Prevents identity leakage during paging in multi-network environments.
- **Privacy Protection Mechanisms:** Ensures slice, subscription, and policy data remain confidential.

E.6 Network Availability & Signaling Security

Ensures the continuous availability and reliability of network services by protecting against attacks that disrupt connectivity or signaling procedures.

- **Denial-of-Service Resistance:** Protects against resource exhaustion attacks.
- **Signaling Flood Protection:** Detects and mitigates signaling flood attacks.
- **Rogue Base Station Detection:** Identifies and blocks connections to fake eNodeBs/gNodeBs.
- **Physical Layer Attack Protection:** Defends against RF jamming, replay, and signal spoofing attacks.

E.7 Interworking Security

Ensures secure and consistent behavior when UEs operate across different access technologies, network generations, and specification layers.

- **Specification Downgrade Protection:** Blocks fallback to insecure or legacy specifications.
- **Cross-Specification Interaction Security:** Prevents vulnerabilities caused by NAS, RRC, SBA, and IP interactions.
- **Specification Compatibility:** Ensures secure, consistent behavior across 4G, 5G, NSA, and SA deployments.

E.8 Threat Detection & Logging

Enables real-time anomaly detection and comprehensive monitoring to ensure system integrity.

- **Cross-Layer Threat Detection:** Monitors interactions across NAS, RRC, SBA, and user-plane layers.
- **Logging & Auditing:** Maintains secure, tamper-proof logs for incident analysis and forensic investigations.
- **Timer Behavior Verification:** Validates specification timers to prevent potential exploitation.
- **Counter Behavior Verification:** Ensures NAS, RRC, and PDCP

Table 8: Test case for validating `service integrity` violation in Node 2073

Direction legend. The U–M column indicates the signaling direction between the UE and the mobile network: ⟶ denotes a UE-to-network message, ⟵ denotes a network-to-UE message, and "-" denotes a local state transition or a step without an explicitly exchanged message.					
Step	**Procedure**	**U–M**	**Message**	**Parameter**	**Verdict**
1	The UE is switched on	-	-	-	-
2	The UE initiates the registration procedure	⟶	REGISTRATION REQUEST	initial registration	-
3	The network accepts the registration and UE enters 5GMM-CONNECTED	⟵	REGISTRATION ACCEPT	-	-
4	The lower layer sends RRC suspend indication to UE	⟵	RRC Connection Suspend	-	-
5	The UE transitions to 5GMM-CONNECTED with RRC inactive indication	-	-	-	-
6	A trigger occurs to update UE radio capability; the UE prepares a new registration	⟶	REGISTRATION REQUEST	NG-RAN-RCU = "UE capability update needed"	-
7	The UE transitions to 5GMM-IDLE and initiates the capability update registration	-	-	-	-
8	The attacker captures the REGISTRATION REQUEST in transit	⟶	(Captured) REGISTRATION REQUEST	-	-
9	The attacker replays the REGISTRATION REQUEST to the network	⟶	REGISTRATION REQUEST	-	-
10	The AMF processes the replayed REGISTRATION REQUEST without sufficient authentication, causing the existing UE connection or registration context to be released, replaced, or disrupted	-	-	Existing UE context affected	Fail

counters are strictly monotonic, unique, and correctly synchronized between UE and network. Detects and prevents counter wraparound, reuse, or desynchronization.

E.9 Regulatory Compliance

Ensures adherence to legal, regulatory, and industry security requirements while protecting against fraudulent activities.

• **Regulatory Compliance:** Enforces lawful intercept, data protection, and privacy regulations.
• **Billing & Fraud Protection:** Prevents manipulation of charging data and unauthorized resource usage.

F Test Case Example

Table 8 presents the example test case, which is designed to validate the `service integrity` violation identified in Node 2073.